



Nash equilibrium strategy solving in two-player imperfect-information games: a survey

Huale Li¹ · Zhaobin Wang¹ · Zhen Yang¹ · Shuhan Qi² · Jiajia Zhang²

Received: 17 September 2025 / Accepted: 26 February 2026 / Published online: 25 March 2026
© The Author(s) 2026

Abstract

Imperfect-information games (IIGs) have attracted significant attention due to the complexity arising from hidden information. In recent years, efficiently solving optimal strategies in IIGs has become a major challenge. Equilibrium strategies, with their solid theoretical foundation and demonstrated performance, have achieved great success. The agents like Libratus and DeepNash demonstrate superior performance in public evaluations. This survey comprehensively reviews two key methods for solving Nash equilibrium strategies in two-player zero-sum IIGs: counterfactual regret minimization (CFR) and fictitious self-play (FSP), including their improved variants. We provide a detailed overview from multiple dimensions, including problem modeling, solution paradigms, and classical solving methods. To further evaluate these methods, we conducted a series of evaluation experiments to deeply analyze and compare their performance. Finally, this survey summarizes current challenges and outlines future research directions, aiming to provide useful insights for advancing the field.

Keywords Imperfect-information game · Nash equilibrium · Counterfactual regret minimization · Fictitious self-play

✉ Shuhan Qi
shuhanqi@cs.hitsz.edu.cn

✉ Jiajia Zhang
zhangjiajia@hit.edu.cn

Huale Li
lihuale@lzu.edu.cn

Zhaobin Wang
zhaobin_wang@hotmail.com

Zhen Yang
zhenyang@lzu.edu.cn

¹ School of Information Science and Engineering, Lanzhou University, Lanzhou 730000, Gansu, China

² School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China

1 Introduction

Games have served as benchmarks for progress in artificial intelligence for a long time. They can be divided into two categories based on whether the participants can observe all the game information: perfect-information games (PIGs) and imperfect-information games (IIGs). PIGs refer to games where participants can observe all the game information during the process, such as Gomoku, checkers, chess, and Go. In recent years, PIGs' research has achieved great success (Mnih et al. 2015; Silver et al. 2016, 2017, 2018; Jaderberg et al. 2019; Schrittwieser et al. 2020; Ecoffet et al. 2021). Particularly in 2016, AlphaGo (Silver et al. 2016) developed by Google's DeepMind defeated Go world champion Lee Sedol, sparking significant interest in the field of artificial intelligence. Subsequently, DeepMind developed AI systems like AlphaGo Zero (Silver et al. 2017), AlphaZero (Silver et al. 2018), and MuZero (Schrittwieser et al. 2020), which have all reached or even surpassed the level of top human experts. IIGs refer to games where participants can only observe partial information, such as Stratego, bridge, and poker. Compared to PIGs, IIGs involve hidden information, and factors such as incomplete, unreliable, and asymmetric information can lead to exponential growth in the time and space complexity of the game, making research in this area more difficult and challenging. Decision-making techniques related to IIGs can be applied in fields like auction mechanism design, advertising pricing, and securities trading.

Research on IIGs encompasses multiple aspects, including assessing the degree of uncertainty in game information, designing game mechanisms, evaluating game risks, modeling opponents, and solving game strategies. Among these, game strategy holds a pivotal position as it is crucial for players to obtain better strategies for decision-making in IIGs. Currently, the pursuit of solving strategies for IIGs converges primarily on three key directions: knowledge-driven methods aimed at solving approximate Nash equilibria; data-driven methods based on deep learning and reinforcement learning (LeCun et al. 2015; Sutton and Barto 2018); and hybrid methods that integrate knowledge and data, leveraging machine learning algorithms within the framework of game theory. Of particular note, solving equilibrium strategies for IIGs has garnered significant attention from researchers in recent years (Brown and Sandholm 2018; Perolat et al. 2022; Zhang et al. 2022; Li et al. 2021; Perrin et al. 2020; Zhang et al. 2023; Xu et al. 2023; Chen et al. 2023b). The classical approach for solving equilibrium strategies in IIGs is to compute or approximate Nash equilibrium strategy of the game. In games, Nash equilibrium strategies ensure that no participant can improve their revenue by unilaterally switching strategies (Nash et al. 1950). Extensive research has been conducted on Nash equilibrium strategies for solving two-player IIGs across diverse scenarios (Zinkevich et al. 2007; Bowling et al. 2015; Moravčík et al. 2017; Brown and Sandholm 2018, 2019b; Perolat et al. 2022; Chen et al. 2022; Panageas et al. 2024). For example, in 2017, Libratus (Brown and Sandholm 2018) and DeepStack (Moravčík et al. 2017) defeated professional players in two-player no-limit Texas Hold'em poker, while Pluribus (Brown and Sandholm 2019b) defeated professional players in six-player no-limit Texas hold'em in 2019. More recently, Google's DeepMind team introduced DeepNash (Perolat et al. 2022) in 2022, defeating professional-level human players in Stratego.

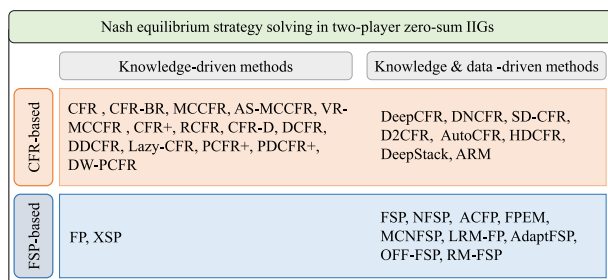
For two-player zero-sum IIGs, counterfactual regret minimization (CFR, Zinkevich et al. (2007)) and fictitious self-play (FSP, Heinrich et al. (2015)) are classical methods to compute or approximate Nash equilibrium strategy. These methods have garnered more and more attention from researchers due to their robust theoretical guarantees of optimal strat-

egy. The successful agents Libratus (Brown and Sandholm 2018), DeepStack (Moravčík et al. 2017) and Pluribus (Brown and Sandholm 2019b) are all based on CFR methods, further validating the efficacy of these equilibrium strategy solving techniques. Since the proposal of these methods, researchers have continuously refined these methods in terms of solving efficiency, solving scale, solving accuracy, and other aspects, leading to the development of several classic refined methods such as CFR+ (Tammelin 2014), DCFR (Brown and Sandholm 2019a), OFF-FSP (Chen et al. 2024), and GFXP (Xu et al. 2025). Despite the substantial research achievements in solving equilibrium strategies for two-player zero-sum IIGs, to our knowledge, a comprehensive and unified introduction and analysis of these methods remain scarce, especially focusing on the methods for solving Nash equilibrium strategies in two-player IIGs (Li et al. 2021; Neller and Lanctot 2013; Ye et al. 2023; Li et al. 2024a).

Therefore, we provide a systematic overview of the methods for solving Nash equilibrium strategies in two-player zero-sum IIGs. We focus on two key techniques CFR and FSP, along with their advanced versions. The taxonomy is visually illustrated in Fig. 1. Our discussion encompasses various dimensions of problem modeling and solution paradigms, as well as traditional solving techniques. Additionally, we introduce high-level agents that utilize these equilibrium strategies and outline the evaluation methods, metrics, and platforms employed for assessing their performance. To enhance the understanding of these techniques, we conducted a series of evaluation experiments, allowing us to analyze and compare their performance characteristics in depth. Ultimately, this survey highlights the current challenges in IIGs while suggesting potential future research directions to address these issues, thereby providing valuable insights for advancing this field. The contributions of this review are summarized as follows:

- *Comprehensive and systematic overview:* This review offers a meticulous and all-encompassing exploration of the methods for resolving Nash equilibrium strategies in two-player zero-sum IIGs. Unlike existing literature that might focus on isolated techniques, our review encompasses a broad spectrum of mainstream methods CFR and FSP, providing insights into both fundamental and advanced versions.
- *In-depth comparative analysis:* To enhance the comprehension of these methodologies, we carried out a series of evaluation experiments. This analysis enables us to compare the performance characteristics of classical CFR techniques and presents a detailed overview of their efficacy and applicability in game scenarios.
- *Identification of current challenges and future directions:* By highlighting the existing challenges in two-player zero-sum IIGs, this survey not only summarizes the current state of the field but also suggests potential avenues for future research. This forward-

Fig. 1 The categorization of Nash equilibrium strategy for IIGs and the corresponding works



looking perspective aims to stimulate advancements in methodologies and applications and facilitate deeper exploration within this area of study.

The organization of this survey is structured as follows. Section 2 introduces Nash equilibrium strategies in two-player games, detailing the game model and corresponding Nash equilibrium strategies. Section 3 discusses CFR-based methods, including the foundational concepts of CFR, classical CFR approaches, and a strategies-solving framework utilizing CFR methods. Section 4 presents the basic FSP method and its associated FSP-based techniques. Section 5 explores high-level agents and methods for solving equilibrium strategies. An experimental analysis of classical methods is provided in Sect. 6. Section 7 outlines existing challenges and suggests directions for future research. Finally, the survey concludes with a summary of the key findings.

2 Nash equilibrium strategies in IIGs

In this section, regarding equilibrium strategies in two-player IIGs, we first present the basic model of the two-player IIG, followed by the definition of Nash equilibrium strategies (Nash et al. 1950). We summarize the primary notations used in this section in Table.1 for the convenience of understanding.

2.1 The model of IIGs

Before solving equilibrium strategies for two-player IIGs, it is crucial to first model the specific game. In games, the normal-form game as a fundamental game model, are usually employed to model simultaneous decision-making problems. A normal-form game is mainly composed of players, action sets and payoff matrices. In a game, players make actions simultaneously and receive the given rewards based on the payoff matrix. For example, in a two-player game 'Rock-Paper-Scissors', there are two players player 1 and player 2 and three available actions 'Rock', 'Paper' and 'Scissor'. The payoff matrix is shown in Table.2. A complete game process is: Player 1 selects the action 'Rock' while Player 2 selects the action 'Paper'. According to the payoff matrix, Player 1 gets -1 and Player 2 gets 1 .

It should be noted that in complex games or most real scenarios, sequential-decision games are more common. The extensive-form game is precisely the game model for such problems. In addition, the focus of this survey is on the strategy-solving issues for extensive-form games, for strategy-solving methods of normal-form games, please refer to Li et al. (2024a). Moreover, unless otherwise specified, the game models mentioned in the text are all assumed to be extensive-form game models.

The extensive-form game model is a classic representation for sequential-decision games, particularly in the domain of IIGs involving private information. These games are typically illustrated using game trees, where nodes (also called decision points) represent game states, as shown in Fig. 2. In the field of IIGs, an extensive-form game can be formally defined as a tuple (N, H, P, σ, u, I) (Osborne and Rubinstein 1994). N is a finite set of players (for two-player IIGs, $N = 2$). H is a set of sequences (called histories) such that: The empty sequence $\emptyset \in H$. If $h \in H$ and $a \in A(h)$, then $(h, a) \in H$. $P : H \setminus Z \rightarrow N \cup \{c\}$ is a player function that assigns to each non-terminal history a player who takes an action after that history or

Table 1 Important notations used in Sect. 2

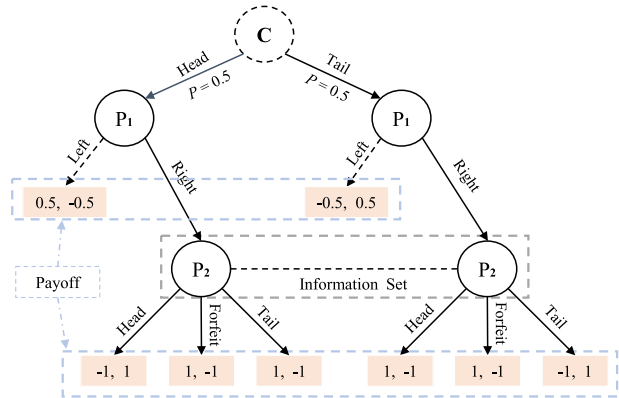
Notations	Detailed descriptions
H	a set of histories, with each history representing a game state and consisting of a sequence of actions taken by all players from the beginning of the game. $h \in H$
I	the information set, representing the set of states that player cannot distinguish
σ_i	the strategy of player i , a probability distribution over actions to each history h
	σ_{-i} are strategies of all players other than player i
σ_i^*	the best response for player i given the strategies σ_{-i}
$\sigma_i(I, a)$	the probability of player i taking action a in an information set I
σ	a strategy profile is a specification of strategies for all players
u	assigns a payoff for every player at each terminal history
$r_{i,t}(I, a)$	immediate regret of not choosing action a in an information set I for player i at the t iteration. $R_{i,t}(I, a)$ is total regret
$R_{i,t}(I, a)$	the total regret for player i when choosing action a at information set I in the t iteration. $R_{i,t}^+(I, a) = \max\{R_{i,t}(I, a), 0\}$
$v_i^{\sigma(I)}$	counterfactual value of player i in information set I on the t iteration under a given strategy profile $\sigma(I)$
$\pi_i^{\sigma}(h)$	the reach probability of player i , refers to the probability of reaching the game state h when all players follow the strategy σ . $\pi_{-i}^{\sigma}(h)$ refers to the contribution to the probability of reaching the game state h from all participants other than participant i (including chance participants) when following the strategy σ

Table 2 The payoff matrix of 'Rock-Paper-Scissors'

Player1	Player2		
	Rock	Paper	Scissor
Rock	(0, 0)	(-1, 1)	(1, -1)
Paper	(1, -1)	(0, 0)	(-1, 1)
Scissor	(-1, 1)	(1, -1)	(0, 0)

c for chance. $Z \subset H$ is a set of terminal histories. $\sigma : \{h \in H \mid P(h) = c\} \rightarrow \Delta(A(h))$ is a function that assigns a probability distribution over actions to each history where chance plays. $A(h)$ denotes the set of actions available after history h , and $\Delta(A(h))$ denotes the set of probability distributions over $A(h)$. $u : Z \rightarrow \mathbb{R}^N$ is a utility function that assigns a payoff to each player for each terminal history. For a two-player zero-sum game, $u_1 + u_2 = 0$. I is an information partition, which for each player partitions the set of non-terminal histories where the player is to move into information sets. The player cannot distinguish between the histories within the same information set.

Fig. 2 The game tree of the game Coin Toss (Brown and Sandholm 2017). "C" represents a chance node. P_1 and P_2 are the game participants. A coin is flipped and lands either Heads or Tails with equal probability, but only player P_1 can see the outcome. The information set of P_2 is depicted by the dotted line between the two P_2 nodes, indicating that player P_2 cannot distinguish between the two states



In the extensive-form game model's game tree, non-terminal nodes show legal actions via directed edges connecting parent and child nodes, depicting state transitions. Terminal nodes indicate game conclusion with payoff distribution. A game process is a path from root to leaf, determined by participants' actions. The model includes a chance player for random events, determined by specific probabilities (e.g., dice rolls). As illustrated, player P_1 chooses between Left (immediate payoff) and Right (leading to P_2 guessing a coin's outcome, with rewards of -1 for P_1 and 1 for P_2 if correct).

2.2 Nash equilibrium in IIGs

Nash equilibrium is an important concept in computer game theory, especially in two-player zero-sum IIGs. In two-player IIGs, Nash equilibrium strategies are generally considered optimal strategies. A Nash equilibrium is a strategy profile σ^* where each player's strategy in the game is a best response. In other words, a Nash equilibrium is a strategy profile where no player can unilaterally switch to another strategy to increase their own payoff. Although finding an equilibrium can be challenging, it has been proven that a Nash equilibrium exists in all finite games and many infinite games.

A best response is a strategy that maximizes a player's payoff given the strategies chosen by the other players. Formally, it is defined as follows (Osborne and Rubinstein 1994): In a strategic game with n players (in two-player IIGs, $n = 2$), let Σ_i be the strategy set of player i and $u_i(\sigma_1, \sigma_2, \dots, \sigma_n)$ be the payoff function of player i given the strategies $(\sigma_1, \sigma_2, \dots, \sigma_n)$. The best response of player i to the strategy profile σ_{-i} of the other players is a strategy $\sigma_i^* \in \Sigma_i$ such that:

$$u_i(\sigma_i^*, \sigma_{-i}) \geq u_i(\sigma_i, \sigma_{-i}), \quad \text{for all } \sigma_i \in \Sigma_i \quad (1)$$

where σ_{-i} denotes the strategies of all players other than player i . In other words, σ_i^* is the best response if it yields the highest possible payoff for player i given the strategies σ_{-i} of the other players. A Nash equilibrium is a strategy profile where each player's strategy is a best response to the strategies of the other players. Formally, a strategy profile $(\sigma_1^*, \sigma_2^*, \dots, \sigma_n^*)$ is a Nash equilibrium if for every player i (Osborne and Rubinstein 1994):

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*), \quad \text{for all } \sigma_i \in \Sigma_i \quad (2)$$

where σ_{-i}^* denotes the best response of all players other than player i .

This implies that no player can unilaterally deviate from their equilibrium strategy to attain a superior payoff. Consequently, in a Nash equilibrium, each player's strategy constitutes the optimal response to the strategies adopted by others. In two-player zero-sum games, adopting a Nash equilibrium strategy ensures, from an expected value perspective, that a player will not incur a loss, irrespective of the opponent's strategy. Hence, a Nash equilibrium strategy in such games is unassailable. Moreover, computing an exact Nash equilibrium for large IIGs is impractical. Therefore, approximate Nash equilibrium, or ϵ -Nash equilibrium, are typically pursued. These strategies permit minor deviations from the exact equilibrium condition, where the potential gain from unilateral strategy changes does not exceed a small threshold ϵ . A strategy profile $(\sigma_1^*, \sigma_2^*, \dots, \sigma_n^*)$ is an ϵ -Nash equilibrium if for every player i (Osborne and Rubinstein 1994), $u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) - \epsilon$, for all, $\sigma_i \in \Sigma_i$, where ϵ is a small positive number.

3 CFR based methods

In two-player zero-sum IIGs, Nash equilibrium ensures that participants, under the premise of rationality, can achieve optimal payoffs when adopting Nash equilibrium strategies. In recent years, there have been two main types of methods for solving Nash equilibrium and approximate Nash equilibrium: methods centered on counterfactual regret minimization (CFR, Zinkevich et al. (2007)), and methods centered on fictitious self-play (FSP, Heinrich et al. (2015)). In this section, we will introduce CFR methods and its subsequent improvements.

3.1 Counterfactual regret minimization

Since CFR was proposed by Zinkevich et al. (2007), it has gradually become a classic method for solving Nash equilibrium strategies in two-player zero-sum IIGs with perfect recall. CFR solves strategies using the Regret Matching (RM, Foster and Vohra (1999)) algorithm, which is a simple no-regret learning algorithm widely applied in solving IIGs. In sequential decision-making games, the regret measures the cumulative reward difference between an agent's acting strategy and the best fixed action in hindsight. This metric underpins RM, a standard online learning algorithm that updates action probabilities proportionally to accumulated positive regret. By normalizing regrets across actions, RM ensures that actions yielding higher historical regret gain increased selection probability in subsequent rounds. However, RM faces computational limitations in complex extensive-form IIGs. CFR addresses this by decomposing overall regret into local counterfactual regrets at each information set.

CFR mainly consists of three steps: total regret calculation, strategy update, and average strategy calculation. Regret calculation is to compute the regret value for each game action at each information set during each iteration. Strategy update is to update the iterative strategy for the next iteration through the RM. The three steps are introduced as following.

3.1.1 Total regret calculation

First, let σ^t be the iterative strategy at the t iteration. The immediate regret $r_{i,t}(I, a)$ for game action a at information set I during the t iteration can be formally defined as:

$$r_{i,t}(I, a) = v_i^{\sigma^t}(I, a) - v_i^{\sigma^t}(I) \quad (3)$$

where $v_i^{\sigma^t}(I)$ is the counterfactual value of player i at information set I during the t iteration, representing the expected payoff of player i assuming information set I has been reached, multiplied by a weight, which is the probability of reaching game state h by all players except player i . The formal definition of $v_i^{\sigma}(I)$ is:

$$v_i^{\sigma}(I) = \sum_{h \in I} (\pi_{-i}^{\sigma}(h) \sum_{z \in Z} (\pi^{\sigma}(h, z) u_i(z))) \quad (4)$$

where $u_i(z)$ is the payoff of player i at leaf node z . The reach probability $\pi^{\sigma}(h)$ refers to the probability of reaching the game state h when all game participants follow the strategy σ . The formal definition of $\pi^{\sigma}(h)$ is: $\pi^{\sigma}(h) = \prod_{h' \cdot a' \sqsubseteq h} \sigma(h', a')$. The term $\pi_i^{\sigma}(h)$ refers to the contribution of participant i to the probability of reaching the game state h when all participants follow the strategy σ . The formal definition of $\pi_i^{\sigma}(h)$ is given by $\pi_i^{\sigma}(h) = \prod_{h' \cdot a' \sqsubseteq h | P(h')=i} \sigma(h', a')$. On the other hand, $\pi_{-i}^{\sigma}(h)$ refers to the contribution to the probability of reaching the game state h from all participants other than participant i (including chance participants) when following strategy σ . The formal definition of $\pi_{-i}^{\sigma}(h)$ is $\pi_{-i}^{\sigma}(h) = \prod_{h' \cdot a' \sqsubseteq h | P(h') \neq i} \sigma(h', a')$. $\pi^{\sigma}(h, z) = \pi^{\sigma}(z) / \pi^{\sigma}(h)$ if $h \sqsubseteq z$, and zero otherwise. Further, the reach probability of participant i at the information set I_i , denoted as $\pi_i^{\sigma}(I_i)$, is formally defined as: $\pi_i^{\sigma}(I_i) = \prod_{I'_i \cdot a' \sqsubseteq I_i} \sigma(I'_i, a')$. The reach probability of other participants at the information set I_i , excluding participant i , denoted as $\pi_{-i}^{\sigma}(I_i)$, is formally defined as: $\pi_{-i}^{\sigma}(I_i) = \sum_{h \in I_i} \pi_{-i}^{\sigma}(h)$.

Additionally, the formal definition of the counterfactual value $v_i^{\sigma}(I, a)$ for player i choosing action a at information set I is:

$$v_i^{\sigma}(I, a) = \sum_{h \in I} (\pi_{-i}^{\sigma}(h) \sum_{z \in Z} (\pi^{\sigma}(h \cdot a, z) u_i(z))) \quad (5)$$

The formal definition of total regret $R_{i,T}(I, a)$ for choosing action a at information set I in the T iteration is:

$$R_{i,T}(I, a) = \sum_{t=1}^T r_{i,t}(I, a) \quad (6)$$

usually, we use $R_{i,T}^+(I, a) = \max\{R_{i,T}(I, a), 0\}$. The regret for player i in the entire game is formally defined as:

$$R_{i,T} = \max_{\sigma'_i \in \Sigma_i} \sum_{t=1}^T (u_i(\sigma'_i, \sigma_{t,-i}) - u_i(\sigma_{t,i}, \sigma_{t,-i})) \quad (7)$$

where Σ_i is the set of pure strategies for player i .

3.1.2 Strategy update

CFR iteratively updates strategies using the RM. The distribution of actions chosen by player i in information set I is proportional to positive regret values. The probability of player i choosing action a at iteration $t + 1$ is formally defined as:

$$\sigma_i^{t+1}(I, a) = \begin{cases} \frac{R_{i,t}^+(I, a)}{\sum_{a' \in A(I)} R_{i,t}^+(I, a')} & \text{if } \sum_{a'} R_{i,t}^+(I, a') > 0 \\ \frac{1}{|A(I)|} & \text{otherwise} \end{cases} \quad (8)$$

When updating the iterative strategy, two scenarios are considered. When the total regret is greater than 0, the update follows the RM. When the total regret is less than or equal to 0, all legal actions are chosen with equal probability.

3.1.3 Average strategy calculation

CFR ensures that as the number of iterations increases, its average strategy will eventually be a Nash equilibrium or an approximate Nash equilibrium. The average strategy of player i at T iteration in information set I is formally defined as:

$$\bar{\sigma}_i^T(I) = \frac{\sum_{t=1}^T (\pi_i^{\sigma^t}(I) \sigma_i^t(I))}{\sum_{t=1}^T \pi_i^{\sigma^t}(I)} \quad (9)$$

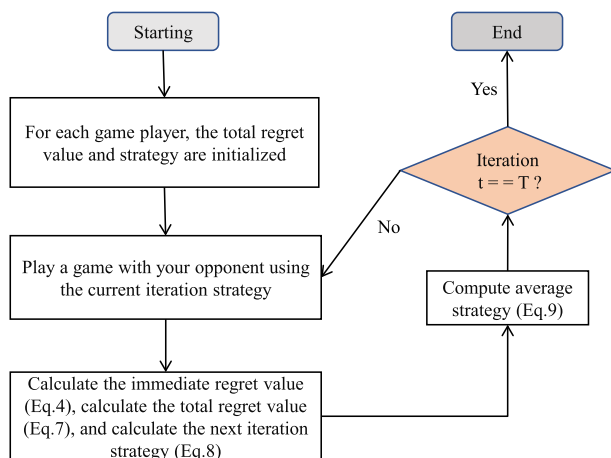
In summary, the complete process for solving the Nash equilibrium strategy using CFR in two-player IIGs is as follows: First, initialize the regret values, total regret values, and iterative strategies. Second, participants play the game using the iterative strategies. Third, calculate the corresponding regret values and total regret values with Eq. 3 and Eq. 6, and update the iterative strategies for the next iteration with Eq. 8. Then, calculate the average strategy with Eq. 9. Finally, check if the number of iterations has reached the maximum set value. If it has, the solution process ends; if not, repeat steps 1-2. The process of solving Nash equilibrium strategy by CFR is as shown in Fig. 3.

Following the above process, CFR can theoretically ensure that in a two-player zero-sum IIG, the final average strategy obtained is either the Nash equilibrium strategy or an approximate Nash equilibrium strategy.

3.2 Strategy solving framework of CFR

As previously mentioned in 3.1, traditional CFR require a complete expansion of the game tree, which significantly limits CFR's application in directly solving large-scale two-player zero-sum IIGs. When using CFR to address larger-scale two-player zero-sum IIGs, researchers commonly adopt the "abstraction-solving-mapping" framework to solve the strategy. This framework contains three steps described as follows.

Step 1: Game abstraction. This phase primarily involves using abstraction algorithms to simplify the original game into a smaller-scale game. Large-scale IIGs are primarily due

Fig. 3 The process of CFR solving Nash equilibrium strategy

to the extensive size of the game state space and action space. Therefore, abstraction algorithms (Johanson et al. 2013; Basak 2016; Hawkin et al. 2011; Shi and Littman 2000; Gilpin and Sandholm 2008; Teófilo et al. 2013; Gilpin and Sandholm 2007) typically compress these two spaces. Typical abstraction methods include using clustering algorithms (Blashfield and Aldenderfer 1988) to compress similar game states and discretizing continuous actions (Hawkin et al. 2011; Brown et al. 2015) to abstract the action space.

Step 2: Strategy solving. In this phase, CFR is directly used to solve the abstracted game. Since the size of the abstracted game is already within the solvable range of CFR, CFR can be directly employed for strategy solving.

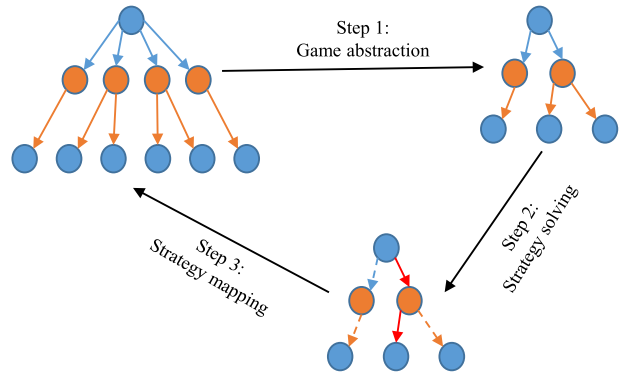
Step 3: Strategy mapping. This phase involves mapping the game strategy obtained through CFR back to the original game. Because the strategy solved by CFR pertains to the abstracted game, whereas practical applications require the strategy for the original game, strategy mapping is essential. Strategy mapping must satisfy conditions such as boundary constraints, monotonicity, scale invariance, and boundary robustness (Ganzfried and Sandholm 2013).

Figure 4 provides a comprehensive depiction of the "abstraction-solving-mapping" strategy-solving framework. It is evident that after the first step, "Game Abstraction," the nodes and arrows in the game tree on the upper right, which represent the game problem, are significantly reduced. This reduction indicates a compression in the scale of the problem. Furthermore, in the game tree at the bottom, the appearance of a solid red arrow signifies the result of the second step, "Strategy Solving," illustrating the derived game strategy. Through this solution framework, it can be found that the abstract algorithm, the solving scale of CFR, and the strategy mapping algorithm all have an impact on the final strategy. For CFR itself, its larger problem-solving scale and higher strategy accuracy are all beneficial to the final strategy.

3.3 Classical CFR-based methods (without neural networks)

Since CFR was proposed Zinkevich et al. (2007), it has shown great performance in solving Nash equilibrium strategies in two-player zero-sum IIGs, attracting much attention from researchers and leading to the development of various improvement methods. In this sec-

Fig. 4 The abstraction-solving-mapping framework. The tree structure illustrates the game tree of the problem, with different colored circles representing distinct game participants and different colored arrows indicating the corresponding actions of these participants. The large game tree represents the original problem, whereas the smaller game tree signifies the abstracted and reduced-scale version of the game



tion, we briefly introduce a selection of classic improvement methods that followed CFR, focusing on their improvement ideas and how they differ from the traditional CFR.

3.3.1 Monte Carlo CFR (MCCFR) & VR-MCCFR

From the introduction to CFR in section 3.1, it becomes apparent that CFR, when solving equilibrium strategies in IIGs, first models the game as an extensive-form game and represents it using a game tree. In each iteration, it requires a complete traversal of the nodes in the game tree. CFR is time-consuming, and while it is feasible to traverse and solve strategies for small-scale game trees, the question arises as to how large-scale or even ultra-large-scale game trees can be directly solved.

To address the aforementioned issue, Lanctot et al. (2009) improved CFR by incorporating Monte Carlo sampling techniques (Shapiro 2003). The core concept of MCCFR involves estimating and updating regret values via Monte Carlo sampling to mitigate computational complexity and improve efficiency. MCCFR selectively samples a subset of decision paths in each iteration, updating strategies based on these sampled outcomes. This approach leverages random sampling to approximate the behavior of the entire decision tree, thereby effectively approximating optimal strategies without necessitating exhaustive traversal of the decision tree.

In MCCFR (Lanctot et al. 2009), two sampling types are used: external-sampling (ES) and outcome-sampling (OS). Before introducing these two sampling types, we first need to clarify a concept block. In MCCFR, block Q sampling entails partitioning the set of terminal histories Z into subsets known as blocks. During each iteration, a single block is sampled, and updates are applied exclusively to the terminal histories within the selected block. This approach mitigates computational complexity by concentrating on a more manageable subset of terminal histories at any given time. Let $Q = \{Q_1, Q_2, \dots, Q_r\}$ be the set of blocks, with each block $Q_j \subseteq Z$. The probability of considering block Q_j is q_j , where $\sum_{j=1}^r q_j = 1$. For a terminal history, the probability of considering $q(z) = q_j$, where $z \in Q_j$. The sampled counterfactual value when updating block j is:

$$\tilde{v}_i(\sigma, I|j) = \sum_{z \in Q_j \cap Z_I} \frac{u_i(z) \pi_i^\sigma(z[I]) \pi^\sigma(z[I], z)}{q(z)} \quad (10)$$

ES-MCCFR: External sampling in MCCFR focuses on sampling the actions of the opponent and chance nodes. This method avoids traversing the entire game tree by sampling the actions external to the current player, which reduces the number of nodes to be traversed and updated. For each information set I and action a , the sampled counterfactual regret is, $\tilde{r}(I, a) = (1 - \sigma(a|I)) \sum_{z \in Q \cap Z_I} u_i(z) \pi_i^\sigma(z[I]a, z)$, where $\sigma(a|I)$ is the probability of taking action a in information set I according to the current strategy σ . Z_I is the subset of all terminal histories where a prefix of the history is in I ; for $z \in Z_I$, $z[I]$ is that prefix. $\sigma(a|I)$ and $\sigma(I, a)$ represent the same meaning.

OS-MCCFR: Outcome sampling in MCCFR involves sampling a single trajectory through the game tree based on the current strategy profile. Only the actions and information sets along this sampled trajectory are updated. This method significantly reduces computational burden by focusing on a single path rather than the entire tree. For each information set I and action a , the sampled counterfactual regret is:

$$\tilde{r}(I, a) = \begin{cases} w_I \cdot (1 - \sigma(a|z[I])) & \text{if } (z[I]a) \preceq z \\ -w_I \cdot \sigma(a|z[I]) & \text{otherwise} \end{cases} \quad (11)$$

where $w_I = \frac{u_i(z) \pi_{-i}^\sigma(z) \pi_i^\sigma(z[I]a, z)}{\pi^{\sigma'}(z)}$ and $\pi^{\sigma'}(z)$ is the probability of the sampled terminal history z occurring under the sampling profile σ' .

VR-MCCFR (Schmid et al. 2019): Variance Reduction MCCFR (VR-MCCFR) addresses the high variance problem faced by MCCFR when dealing with large-scale, complex extensive-form games. While MCCFR reduces computational costs by sampling portions of the game tree, this approach introduces significant variance, causing substantial fluctuations in the estimates and negatively impacting the algorithm's convergence speed. Specifically, MCCFR may exhibit rapid short-term convergence, but due to the high variance, its long-term convergence is slow, limiting its practical performance.

VR-MCCFR enhances MCCFR's performance by incorporating baseline values and control variates to reduce the variance in each iteration. The key improvements includes three parts. Use of Baseline Values: State-action baseline values are employed to adjust sampled values when estimating counterfactual values, thereby reducing estimate volatility. Recursive Bootstrapping: In each iteration, current estimates are guided by other estimates using recursive bootstrapping, propagating the benefits of variance reduction throughout the entire sampled trajectory. Introduction of Control Variates: Control variates are introduced to maintain unbiased estimates while significantly reducing variance. These enhancements enable VR-MCCFR to effectively mitigate the high variance issue in MCCFR, substantially accelerating convergence speed and improving performance in practical applications.

3.3.2 CFR+

CFR+ (Tammelin 2014) introduces an enhanced version of vanilla CFR by employing a new regret-matching variant called regret-matching+. This approach enhances computational efficiency and memory usage, thereby facilitating faster convergence to approximate Nash equilibrium solutions. In CFR+, the regret-matching+ algorithm considers only positive total regrets, a modification that accelerates convergence and prevents the accumulation of negative regrets, thus improving algorithmic efficiency. Unlike CFR, CFR+ does not necessitate averaging over the entire strategy profile for convergence to an approximate

Nash equilibrium. However, weighted averaging can still be employed to further optimize the convergence rate. Additionally, by reducing the number of total regret values that need to be stored (as many become zero), CFR+ achieves higher compression ratios, significantly lowering the memory footprint.

Based on the above analysis, we present the formulas for calculating total regret and updating iterative strategies in CFR+, which are main differences from CFR. The alternative counterfactual regret values $Q_i^t(I, a)$ updated as follows:

$$Q_i^t(I, a) = \left(Q_i^{t-1}(I, a) + R_{i,t}(I, a) - R_{i,t-1}(I, a) \right)^+ \quad (12)$$

where $Q_i^0(I, a) = 0$, and $Q_i^t(I, a)$ updates reflect counterfactual regret changes, with negative values clipped to 0. The next iteration strategy $\sigma_i^{t+1}(I, a)$ is calculated as:

$$\sigma_i^{t+1}(I, a) = \begin{cases} \frac{Q_i^t(I, a)}{\sum_{a' \in A} Q_i^t(I, a')}, & \text{if } \sum_{a' \in A} Q_i^t(I, a') > 0 \\ \frac{1}{|A|}, & \text{otherwise} \end{cases} \quad (13)$$

3.3.3 Discounted CFR (DCFR) & DDCFR

DCFR (Brown and Sandholm 2019a) introduces several key enhancements to vanilla CFR, including discounting regrets, weighted averaging, and optimistic regret matching. In DCFR, the technique of discounting regrets involves applying different discount factors to positive and negative regrets, typically reducing the influence of regrets from earlier iterations. This adjustment helps to prioritize more recent decisions, which are likely to be more relevant as the strategy converges.

For weighted averaging, unlike vanilla CFR, which assigns equal weight to all iterations when calculating the average strategy, DCFR employs a weighted approach. This method assigns greater significance to more recent iterations, thereby enhancing the rate of convergence. Additionally, some variants of DCFR incorporate optimistic regret matching techniques. These techniques temporarily amplify the impact of recent iterations, further expediting the convergence process.

For total regrets at information set I for action a up to iteration t :

$$R_{i,t}(I, a) = \begin{cases} R_{i,t-1}(I, a) \frac{(t-1)^\alpha}{(t-1)^\alpha + 1} + r_{i,t}(I, a), & \text{if } R_{i,t-1}(I, a) > 0 \\ R_{i,t-1}(I, a) \frac{(t-1)^\beta}{(t-1)^\beta + 1} + r_{i,t}(I, a), & \text{otherwise} \end{cases} \quad (14)$$

The updated strategy for player i at information set I for action a is:

$$\sigma_i^{t+1}(I, a) = \frac{R_{i,t}(I, a)}{\sum_{a' \in A(I)} R_{i,t}(I, a')} \quad (15)$$

if all $R_{i,t}(I, a')$ are zero, then $\sigma_i^{t+1}(I, a) = \frac{1}{|A(I)|}$.

For average strategy calculation, the weighted average strategy at information set I for action a up to iteration T :

$$\bar{\sigma}_i^T(I, a) = \frac{\sum_{t=1}^T \left(\frac{t^\gamma}{t^\gamma + 1} \cdot \pi_i^t(I) \sigma_i^t(I, a) \right)}{\sum_{t=1}^T \left(\frac{t^\gamma}{t^\gamma + 1} \cdot \pi_i^t(I) \right)} \quad (16)$$

DDCFR (Xu et al. 2024a): Current variants of CFR, such as CFR+ and DCFR, have limitations in weight allocation as they use fixed, manually specified schemes to weight each iteration, which restricts their potential. To overcome these limitations, Dynamic DCFR (DDCFR) uses a dynamically and automatically learned discount scheme to weight iterations, instead of relying on fixed schemes. The iteration process of CFR is formalized as a carefully designed Markov Decision Process (MDP), transforming the problem of learning the discount scheme into a policy optimization problem within this MDP. Furthermore, DDCFR designs a highly efficient training algorithm based on evolution strategies to optimize the discount policy, endowing it with strong generalization capabilities that enable it to perform well in new games.

3.3.4 AutoCFR

Traditional CFR variants like CFR+, Linear CFR, and DCFR have improved convergence rates over the vanilla CFR, but their manual design through trial and error requires significant effort and expertise. AutoCFR (Xu et al. 2022) addresses these limitations by automating the design of novel CFR algorithms using meta-learning. AutoCFR, a meta-learning framework, leverages evolutionary algorithms to explore a vast combinatorial space of potential CFR variants defined by a specially designed search language. This language is capable of representing many existing CFR variants and facilitates the discovery of new, potentially more efficient algorithms. The primary objective of the AutoCFR framework is to identify novel CFR variants that generalize well across different IIGs, thereby significantly reducing the effort and expertise required in manual algorithm design. AutoCFR primarily consists of the following three steps:

Step 1: Designing a search language. The process begins with the design of a search language that defines input nodes (such as the current strategy, total regret, and iteration number), operation nodes (including basic mathematics, linear algebra, and probability functions), and output nodes (such as the updated strategy and total regret). An initial population of candidate CFR algorithms is generated either randomly or based on existing CFR variants, and each candidate is subsequently evaluated on a set of training games.

Step 2: Evolutionary search algorithm. The core of AutoCFR's implementation is an evolutionary search algorithm. This involves selecting a subset of candidate algorithms from the population, identifying the best-performing one as the parent for mutation, and applying random alterations to create new candidates. These new candidates undergo validation checks to ensure they are valid and not redundant. Additionally, early stopping criteria are applied by evaluating candidates on a simple hurdle game, discarding those that do not meet predefined performance thresholds. Valid and promising new candidates are then added to the population, replacing the oldest members to maintain a fixed population size. This iterative process is distributed across multiple processors to efficiently manage computational demands.

Step 3: Evaluation. The most promising candidate algorithms undergo extensive evaluation on both training and unseen testing games. The algorithm with the highest overall performance score is selected as the final output of the AutoCFR framework. This approach

not only significantly reduces the effort and expertise required but also has the potential to discover more efficient and generalizable CFR variants than those manually designed. By following these steps, AutoCFR automates the design process of CFR algorithms, exploring a wide range of potential configurations to identify those that perform optimally across diverse game scenarios.

DCFR+: AutoCFR discovered a new variant named DCFR+, which combines the strengths of CFR+ and DCFR, yielding superior performance. DCFR+ sets negative total regrets to zero and applies a novel discounting method to improve convergence rates significantly. The total regret in DCFR+ can be updated with the following formula:

$$R_{i,T}(I, a) = \max \left(0, R_{i,T-1}(I, a) \cdot \frac{(T-1)^{1.5}}{(T-1)^{1.5} + 1.5} + r_{i,T}(I, a) \right) \quad (17)$$

The cumulative strategy in DCFR+ is depicted as:

$$C_i^T(I, a) = C_i^{T-1}(I, a) \cdot \frac{T-1}{T} + \pi_\sigma^T(I) \cdot T^3 \cdot \sigma_i^T(I, a) \quad (18)$$

The iteration strategy in DCFR+ can be defined as:

$$\sigma_i^{T+1}(I, a) = \frac{R_{i,T}^+(I, a)}{\sum_{a'} R_{i,T}^+(I, a')} \quad (19)$$

3.3.5 PCFR+ & PDCFR+

The principal innovation of predictive CFR (PCFR+, Farina et al. (2021)) is its application of predictive Blackwell approachability, a concept that integrates forecasted regret vectors into the decision-making framework. PCFR+ builds upon traditional Blackwell approachability by incorporating anticipatory estimates of forthcoming payoff vectors, thereby enhancing the decision-maker's performance contingent on the precision of these predictions. PCFR+ delineates a robust connection between RM, RM+, follow-the-regularized-leader (FTRL), and online mirror descent (OMD), elucidating that RM and RM+ can be interpreted as specific implementations of FTRL and OMD within the Blackwell approachability paradigm. By employing predictive variants of FTRL and OMD, the study introduces predictive RM (PRM) and predictive RM+ (PRM+), thereby refining the implementation approach and optimizing the workflow for solving IIGs.

The process of PCFR+ is essentially similar to CFR, and the core of the algorithm also encompasses regret calculation and strategy updates. For the step regret calculation, the regret value θ_t is:

$$\theta_t = [z_{t-1} + \langle m_t, x_{t-1} \rangle 1 - m_t]_+ \quad (20)$$

where m_t is predicted loss vector and $m_t = \ell_{t-1}$, Δ_n is a simplex domain, the cumulative loss $r_t = r_{t-1} + \langle \ell_t, x_t \rangle 1 - \ell_t$, the total predictive regret value $z_t = [z_{t-1} + \langle \ell_t, x_t \rangle 1 - \ell_t]_+$.

For the step strategy updates, the next strategy x_t based on the updated regret value is:

$$x_t = \begin{cases} \theta_t / \|\theta_t\|_1 & \text{if } \theta_t \neq 0 \\ \text{arbitrary point in } \Delta_n & \text{if } \theta_t = 0 \end{cases} \quad (21)$$

the average strategy is:

$$x_t^{\text{avg}} = \frac{6}{T(T+1)(2T+1)} \sum_{t=1}^T t^2 x_t \quad (22)$$

PDCFR+ (Xu et al. 2024b): In contrast to PCFR+, PDCFR+ incorporates a temporally weighted regret mechanism that emphasizes more recent iterations. This methodology facilitates the rapid reduction of high regrets resulting from suboptimal actions in earlier iterations. PDCFR+ synergistically integrates the discounting mechanism from DCFR with the predictive techniques of PCFR+. By discounting total regrets while concurrently leveraging predictive updates, this hybrid approach effectively attenuates the adverse effects of dominated actions and enhances the overall convergence speed of the algorithm. In PDCFR+, the regret calculation is defined as:

$$R_{j,t} = \left[R_{j,t-1} \frac{(t-1)^\alpha}{(t-1)^\alpha + 1} + r_{j,t} \right]^+ \quad (23)$$

The strategy can be updated with:

$$x_j^{t+1} = \frac{\tilde{R}_{j,t+1}}{\|\tilde{R}_{j,t+1}\|_1} \quad (24)$$

where $\tilde{R}_{j,t+1} = [R_{j,t} \frac{t^\alpha}{t^\alpha + 1} + v_j^{t+1}]^+$, the average strategy can be calculated by:

$$X^t = X^{t-1} \left(\frac{t-1}{t} \right)^\gamma + \dot{x}^t \quad (25)$$

3.4 Classical CFR-based methods (with neural networks)

Leveraging the rapid advancements in deep learning technologies, integrating CFR with neural networks for strategy computation has emerged as a significant research focus. In this section, the first combining CFR with neural networks, namely DeepCFR, and its improved variants will be introduced.

3.4.1 DeepCFR

In large-scale imperfect-information games, vanilla CFR often relies on abstraction to simplify the game by grouping similar states and treating them identically. This abstraction process, however, requires significant manual intervention and domain-specific knowledge, potentially resulting in coarse approximations of true equilibria. Furthermore, vanilla CFR struggles with the inefficiency of handling extremely large state spaces.

DeepCFR (Brown et al. 2019) addresses these limitations by integrating CFR with deep neural networks. This approach leverages function approximation to handle large imperfect-information games directly, eliminating the need for abstraction. DeepCFR employs deep learning to approximate policies and value functions, enabling efficient processing of vast state spaces. Additionally, it utilizes MCCFR (Lanctot et al. 2009) with external sampling to reduce computational load by traversing only portions of the game tree at each iteration. Consequently, DeepCFR offers a more scalable and effective solution for solving large-scale games compared to vanilla CFR with abstraction.

As shown in Fig. 5, DeepCFR processes input information sets, which include observed cards and betting history. Cards are represented through a combination of rank, suit, and card embeddings, summed for permutation-invariant sets such as hole, flop, turn, and river. The architecture comprises a card processing branch with three fully connected layers and a betting history branch with two fully connected layers. The output layers generate predicted advantages and action probability logits for each possible action at the given information set.

The process of DeepCFR involves a systematic approach to leveraging deep neural networks for solving large-scale IIGs. The key steps include initialization, game tree traversal, training, and iteration. Here is a concise overview followed by a brief description of each step:

Step 1: Initialization. Initialize the deep neural networks to estimate advantages and store strategy probabilities.

Step 2: Game Tree Traversal. Perform partial traversals of the game tree utilizing ES-MCCFR. During these traversals, strategies are updated based on the predicted regrets generated by the neural networks, allowing the model to handle large state spaces effectively.

Step 3: Training. Collect samples of instantaneous regrets and the corresponding strategy probabilities encountered during the game tree traversals. Use these samples to train the advantage network (for regret estimation) and the policy network (for strategy prediction), minimizing the mean squared error (MSE) between predicted and actual values.

Step 4: Iteration. Repeat the game tree traversal and training steps for a predetermined number of iterations. This iterative process refines the accuracy of the neural networks' predictions, gradually improving the overall strategy.

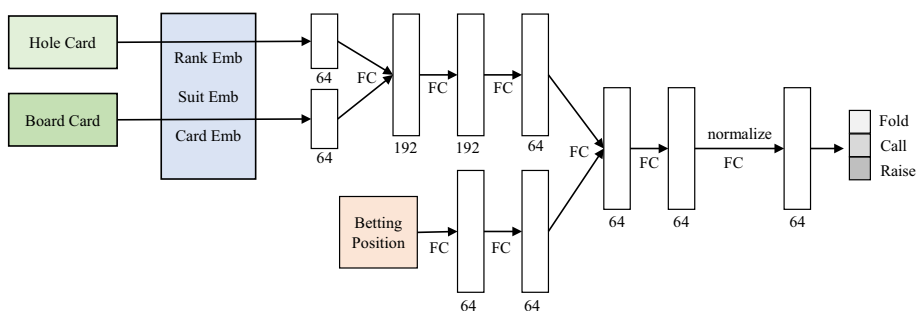


Fig. 5 The neural network architecture of DeepCFR. The network processes an information set, comprising observed cards and betting history, and outputs either advantage values or probability logits for each potential action

3.4.2 DNCFR & SD-CFR & HDCFR

The methods presented here are similar to or are improvements of DeepCFR. In terms of improvements, they all combine neural networks with CFR. Considering that DeepCFR has been introduced in detail earlier, we will provide only a brief introduction to these related methods.

DNCFR (Li et al. 2019): Double neural CFR leverages two distinct neural networks: one dedicated to learning and updating total regrets (regret network) and the other focused on learning and updating average strategies (strategy network). This division of labor enhances training stability and efficiency by allowing each network to specialize in its respective task. Moreover, DNCFR employs advanced sampling techniques, including robust sampling and mini-batch Monte Carlo sampling, which reduce sampling variance and improve memory efficiency. A key feature of DNCFR is its ability to initialize from pre-existing strategies, whether derived from table-based methods or other neural networks, and continuously refine these strategies.

SD-CFR (Steinberger 2019): Single DeepCFR (SD-CFR) streamlines the process of strategy optimization by utilizing a single neural network to approximate value functions directly, thus eliminating the need for a separate network to compute the average strategy. This approach reduces overall approximation errors and computational overhead, leading to more efficient training and reduced complexity. SD-CFR further simplifies the process by storing value networks from past iterations in a buffer, which decreases memory requirements and enables the computation of the average strategy on-the-fly during gameplay without additional training. This combination of methods ensures greater efficiency and accuracy in strategy computation.

HDCFR (Chen et al. 2023a): Hierarchical deep counterfactual regret minimization (HDCFR) represents a significant enhancement over DeepCFR by introducing a hierarchical strategy framework. In HDCFR, high-level strategies orchestrate low-level skills composed of primitive actions. This structure enables the algorithm to leverage prelearned skills for improved decision-making and facilitates skill transfer across different tasks. In contrast, Deep CFR employs a single strategy representation without this hierarchical framework. HDCFR utilizes a low-variance Monte Carlo sampling method combined with a hierarchical baseline function to efficiently estimate counterfactual regrets within large state spaces and deep game trees. This variance reduction technique, absent in Deep CFR, enhances the efficiency of handling complex tasks.

Additionally, HDCFR separates the learning processes for high-level and low-level strategies, ensuring distinct updates and convergence guarantees for each level. This separation enhances scalability, convergence speed, and training efficiency, making HDCFR particularly suitable for tasks with extensive decision horizons and intricate game structures. Conversely, Deep CFR treats the entire strategy as a single entity, thereby lacking the advantages of hierarchical learning and skill transfer.

3.4.3 Strategy solving framework of DeepCFR

This integration enables effective end-to-end strategy solving by utilizing neural networks to estimate strategies and regret values within CFR. Such an approach bypasses the need for

abstraction, thereby minimizing information loss and reducing reliance on domain-specific expertise.

The end-to-end strategy-solving framework primarily involves two steps, as shown in Fig. 6. Firstly, in each iteration, a value network estimates either the immediate regret values or total regret values. Secondly, the RM algorithm is employed to compute the strategy for the subsequent iteration. By iterating these two steps repeatedly, the resulting strategy converges towards a Nash equilibrium, provided that the number of iterations is sufficiently large. Furthermore, a policy network is used to approximate the average strategy over the iterations.

This framework is applied to solve two-player IIGs from input to output in an end-to-end manner. Unlike traditional CFR approaches, which fully expand the game tree, this method approximates regret values using neural networks, akin to a finite-depth search. During the iterative process, both the value network and policy network undergo continuous training as the iterations progress. The value network is specifically tasked with fitting the immediate or total regret values of game actions, while the policy network approximates the average strategy. This results in a robust and scalable solution for complex game-theoretic problems.

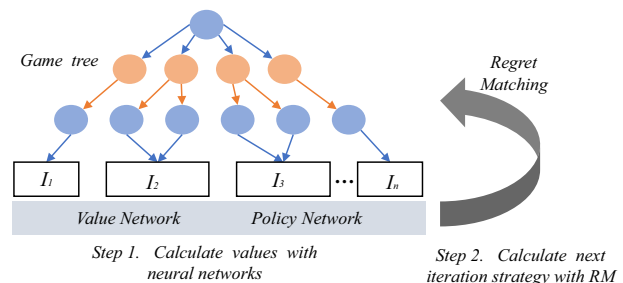
3.5 Other CFR-based methods

Previous sections covered several classical CFR improvements. These methods reflect the field's main optimization directions: enhancing sampling efficiency, optimizing regret matching, integrating function approximation (especially deep learning), and introducing novel algorithmic structures. Following the above four core patterns, this section will categorize and briefly explain other related improvement methods.

Sampling efficiency improvements: A major focus has been on developing more efficient sampling techniques for MCCFR variants. Ponsen et al. (2011) introduced Monte Carlo restricted Nash response (MCRNR), integrating restricted Nash response with MCCFR for rapid convergence to robust best response strategies. Lanctot et al. (2014) presented online outcome sampling (OOS) for MCCFR, enabling efficient online sampling of outcome nodes. Gibson et al. (2012a) addressed slow convergence in multi-action games with average strategy MCCFR (AS-MCCFR), which samples actions based on average strategies.

Regret Matching and pruning optimizations: The work aims to improve the core regret matching process or reduce computation. Brown and Sandholm (2015) introduced regret-based pruning (RBP), enabling CFR to avoid exploring zero-probability paths. Zhou et al. (2019) introduced Lazy CFR, using delayed updates to maintain consistency within disjoint time subsets to improve updating efficiency.

Fig. 6 The end-to-end framework. A game tree for a two-player game is illustrated. Blue circles represent Player 1, and orange circles represent Player 2. Arrows between the circles indicate the legal moves that each player can take. The orange rectangles denote the information sets $I_1, I_2, \dots, I_n$



Integration with function approximation and Deep Learning: With the rise of machine learning, researchers integrated approximation techniques with CFR to handle larger games or learn more complex strategies. Johanson et al. (2011) proposed CFR-BR, which uses best response computation to reduce memory consumption and efficiently approximate the optimal Nash equilibrium strategy. Jin et al. (2018) proposed advantage-based CFR (ARM), learning strategies by updating an approximate advantage function. Liu et al. (2020) introduced neural CFR with Bootstrapping (Neural CFR-B), restoring total regret using surrogate values. Steinberger et al. (2021) combined regret minimization with advantage values in a model-free learning framework. Sychrovský et al. (2023) proposed global CFR, integrating offline meta-learning for global strategy choices. Li et al. (2024b) enhanced strategy accuracy by using deep dueling neural networks to decouple regret values in D2CFR. Ju et al. (2024) presented dynamic weighted pure CFR (DW-PCFR), utilizing pure strategies and addressing dynamic weight complexity. Qiu et al. (2024) proposed VCFR, combining CFR with information gain for Nash equilibrium computation in unknown environments. Wang et al. (2025b) introduced HORSE-CFR, integrating hierarchical opponent reasoning (HOR) and safe exploitation CFR (SE-CFR) to enhance robustness. Wang et al. (2025a) proposed a game-theoretic approach using total regret to construct virtual agents for accelerating autonomous driving testing.

Novel algorithmic structures and theoretical advances: This group includes structural innovations and important theoretical corrections. Burch et al. (2014) introduced CFR-Decomposition (CFR-D), enabling independent solving of the main game and subgames for decomposable games. Waugh et al. (2015) proposed regression CFR (RCFR), using a common regression variable to minimize regret within information sets. Brown and Sandholm (2016) proposed a warm-starting method for CFR, transferring action regret values between scenarios and optimizing parameters with gradient descent. Their subsequent work (Brown and Sandholm 2017) introduced a nested subgame solving technique, which is a core technique of agent "Libratus" (Brown and Sandholm 2018). Burch et al. (2019) provided a corrected proof for CFR+, restoring its original exploitability bound.

Additional relevant studies: Other noteworthy contributions include Gibson et al. (2012b); Abou Risk et al. (2010); Lisý et al. (2015); Liu et al. (2022); Keith and Ahner (2021); Lisy et al. (2016); Swaminathan and Joachims (2015); D'orazio et al. (2019); Kim (2024); Farina et al. (2017, 2020b); Zenati et al. (2023); Farina et al. (2020a); Li et al. (2023), which have also advanced the understanding and application of CFR and related regret minimization techniques.

In summary, recent CFR improvements have focused on sampling techniques, neural network integration, regret matching optimization, and novel structural designs. These enhancements collectively aim to reduce computational complexity, improve strategy precision and generalization, accelerate convergence speed and stability, and expand the scope of solvable games, ultimately enhancing solution efficiency and accuracy.

4 FSP based methods

In 1951, Brown introduced the fictitious play (FP, Brown (1951)) model to approximate Nash equilibrium strategies in two-player zero-sum IIGs. This foundational work spurred further research by numerous scholars on the application of fictitious play in solving strate-

gies for games with imperfect-information. In the fictitious play framework, players alternately make decisions, with each iteration involving a player adopting the optimal response to the opponent's average strategy. It has been demonstrated that, in two-player zero-sum games, the players' average strategies converge to an approximate Nash equilibrium after multiple iterations.

In 1996, Hendon et al. (1996) investigated the use of fictitious play for solving strategies in four specific incomplete information machine game scenarios. Subsequently, in 2002, Hofbauer and Sandholm (2002) examined discrete strategy optimization for two players under conditions of incomplete information using a simplified version of fictitious play. Initial research on fictitious play predominantly remained at a theoretical level, presenting significant challenges for application in concrete machine game strategy solutions (Lambert Iii et al. 2005; Genugten 2000). However, in 2006, Leslie and Collins (2006) introduced the generalized weakened fictitious play method, which permitted the approximation of game strategies. This approach utilized machine learning techniques within the fictitious play framework to approximate Nash equilibrium strategies. This pivotal research provided a new direction for the practical application of fictitious play.

4.1 Fictitious self-play

In 2015, Heinrich et al. (2015) introduced the concept of full-width extensive-form FP (XFP), a method that converts extensive-form games into their corresponding normal-form representations, subsequently employing techniques for solving normal-form games to derive game strategies. This approach effectively addresses game problems with linear complexity. Drawing on the research by Leslie and Collins on generalized weakened fictitious play, Heinrich et al. (2015) proposed a machine learning framework known as fictitious self-play (FSP). FSP integrates machine learning into the fictitious play framework to approximate players' best responses and average strategies. Through iterative play, players generate best responses against the average strategies of their fictitious opponents. With sufficient iterations, the players' average strategies can converge to approximate Nash equilibria in two-player zero-sum games.

In FSP, the iteration solving is a structured procedure aimed at incrementally refining player strategies through three essential steps: data generation, reinforcement learning (RL, Sutton and Barto (2018)), and supervised learning (SL). Initially, data generation is conducted by creating a mixed strategy profile that integrates the average strategy with the best response, guided by a mixing parameter. This profile is used to simulate episodes of the game, which are then stored in memory for subsequent analysis. Following this, RL is employed to derive approximate best responses from the stored episodes. This step involves updating Q-values for state-action pairs, thereby enabling the computation of the best response strategies that maximize these Q-values. Lastly, SL is utilized to update the average strategies based on the stored episodes. This ensures that the learned strategies are realization-equivalent to the average mixed strategies, thus effectively approximating the desired target distribution. By iteratively executing these steps: data generation, reinforcement learning, and supervised learning. The FSP process progressively enhances the strategies, steering them towards Nash equilibria in two-player zero-sum IIGs. The three steps are described as follows:

Step 1: Data generation. This step involves creating a mixed strategy profile by combining the average strategy and the best response using a mixing parameter. Simulated episodes of the game are generated from this mixed strategy profile and the best response profile for each player. These episodes are stored in both reinforcement learning memory and supervised learning memory for further processing. The mixing strategy profile σ_k is computed by:

$$\sigma_k = (1 - \eta_k)\pi_{k-1} + \eta_k\beta_k \quad (26)$$

here, η_k is the mixing parameter that balances the average strategy π_{k-1} and the best response β_k .

In the simulated episodes, generate n episodes from the mixed strategy profile σ_k . For each player i , generate m episodes from the strategy profile (β_i, σ_{-i}) : $D = \{(\text{episode } 1), (\text{episode } 2), \dots, (\text{episode } n)\}$. For each player i : $D_i = \{(\text{episode } 1), (\text{episode } 2), \dots, (\text{episode } m)\}$. These episodes capture the sequence of states, actions, rewards, and next states encountered during the simulations. And then store the generated episodes in the reinforcement learning memory M_i^{RL} , $M_i^{RL} \leftarrow M_i^{RL} \cup D$, and supervised learning memory M_i^{SL} , $M_i^{SL} \leftarrow M_i^{SL} \cup D_i$.

Step 2: RL for best response computation. In this step, the RL memory M_i^{RL} is updated with newly generated episodes. Using RL algorithms like Q-learning (Watkins and Dayan 1992), the approximate best response for each player is computed by updating Q-values based on the transitions observed in the episodes. The best response strategy is then derived from the policy that maximizes these Q-values. Using RL to compute an approximate best response, for each transition tuple $(u_t, a_t, r_{t+1}, u_{t+1})$ in M_i^{RL} , update the Q-value:

$$Q(u_t, a_t) \leftarrow Q(u_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(u_{t+1}, a') - Q(u_t, a_t) \right] \quad (27)$$

here, α is the learning rate, and γ is the discount factor. The best response $\beta_{i,j+1}$ is derived from the policy that maximizes the Q-values, $\beta_{i,j+1}(u) = \arg \max_a Q(u, a)$.

Step 3: SL for strategy updates. Supervised learning memory is updated with the new episodes. The average strategy is learned incrementally by adjusting the policy based on the accumulated counts or weights of actions taken in the sampled episodes. This step ensures that the learned behavioral strategy is realization-equivalent to the average mixed strategy, effectively approximating the target distribution for strategy updates.

Update the SL memory M_i^{SL} with the newly generated episodes. And then use SL to update the average strategy $\pi_{i,j+1}$. This involves learning the policy that approximates the average strategy. For each data tuple (u_t, ρ_t) in M_i^{SL} :

$$N(u_t, a) \leftarrow N(u_t, a) + \rho_t(a) \quad \text{for all } a \in A(u_t) \quad (28)$$

$$\pi(u_t, a) \leftarrow \frac{N(u_t, a)}{N(u_t)} \quad \text{where } N(u_t) = \sum_a N(u_t, a) \quad (29)$$

here, $N(u_t, a)$ is the accumulated count or weight for action a at state u_t .

For the behavioral strategy, ensure the learned behavioral strategy π is realization-equivalent to the average mixed strategy Π :

$$\pi_{i,j+1}(u) = \frac{\sum_{k=1}^n w_k \beta_{i,k}(u)}{\sum_{k=1}^n w_k} \quad (30)$$

where w_k are the weights for each best response $\beta_{i,k}$ in the combination.

4.2 Classical FSP-based methods

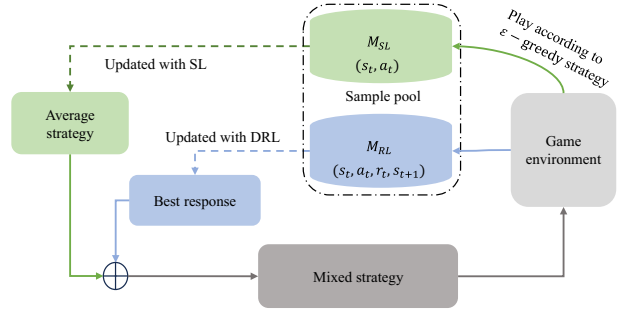
4.2.1 Neural fictitious self-play

Building upon the foundation of FSP, Heinrich and Silver (2016) introduced neural fictitious self-play (NFSP), which presents substantial advancements over FSP in several critical dimensions. Firstly, NFSP amalgamates deep learning and reinforcement learning through the use of neural networks to approximate strategies and value functions, thereby facilitating end-to-end learning in complex and large-scale IIGs without necessitating manually designed abstractions or domain-specific knowledge. Secondly, NFSP incorporates an experience replay mechanism, which stores and replays historical experiences to reduce temporal correlation in samples, thereby enhancing the stability of the learning process. Furthermore, NFSP leverages both supervised learning and reinforcement learning by utilizing two distinct neural networks: one to learn best response strategies and another to learn average strategies, thus concurrently optimizing immediate decisions and long-term strategies. Additionally, NFSP integrates anticipatory dynamics, enabling agents to effectively learn in multi-agent environments by blending average strategies and best response strategies. This integration facilitates more stable convergence to approximate Nash equilibria in dynamic opponent settings. Lastly, the design of NFSP enhances scalability in large-scale imperfect information games, underscoring its potential to address various complex decision-making problems in real-world applications. Collectively, these innovations render NFSP significantly superior to traditional FSP methodologies in terms of learning stability, efficiency, and applicability, thereby achieving more robust approximate Nash equilibrium strategy learning.

The Fig. 7 shows the framework of NFSP. NFSP maintains two distinct sample pools for storing training samples used for the DRL action-value network and the average strategy network, respectively. Notably, the samples for training the average strategy are derived from game data collected while employing the best response. NFSP utilizes deep neural networks to learn the average strategy and employs DRL to learn the best response. In practical self-play of games, a mixed strategy is adopted. This discussion focuses on the selection of the mixed strategy, sample storage, and network updates during the self-play process within NFSP. Other procedures are fundamentally analogous to those in FSP and will not be expounded upon here.

Self-play. At the beginning of each game round, the NFSP agent selects either the ϵ -greedy strategy or the average strategy based on a preset probability. The agent executes an action according to the chosen strategy and observes the current state and its reward. The current strategy σ is defined as:

Fig. 7 The framework of NFSP



$$\sigma = \begin{cases} \epsilon\text{-greedy}(Q), & \text{with probability } \eta \\ \Pi, & \text{with probability } 1 - \eta \end{cases} \quad (31)$$

if using ϵ -greedy strategy (probability η): Choose action a randomly with probability ϵ , $a_t = \text{random_action}()$, or choose action a that maximizes $Q(s, a)$ with probability $1 - \epsilon$, $a_t = \arg \max_a Q(s_t, a)$. If using the average strategy (probability $1 - \eta$): Sample action a from the average policy network Π , $a_t \sim \Pi(s_t, \cdot | \theta_\Pi)$. Observe the initial state s_1 and choose action a_t based on the selected strategy. At each time step t , execute action a_t and observe the resulting state s_{t+1} and reward r_{t+1} .

Execute action and store experience. Store the current state s_t , action a_t , reward r_{t+1} , and next state s_{t+1} into the reinforcement learning memory M_{RL} , $M_{RL} \leftarrow (s_t, a_t, r_{t+1}, s_{t+1})$. If using the ϵ -greedy strategy, store the state-action pair s_t and a_t into the supervised learning memory M_{SL} , $M_{SL} \leftarrow (s_t, a_t)$.

Update neural networks. Sample data from the SL memory to update the average policy network by maximizing the log-probability of past actions. Sample data from the DRL memory to update the action-value network by minimizing the Bellman error. For the average policy network, randomly sample a batch of data from the SL memory M_{SL} and update the average policy network Π 's parameters θ_Π using supervised learning:

$$L(\theta_\Pi) = \mathbb{E}_{(s,a) \sim M_{SL}} [-\log \Pi(s, a | \theta_\Pi)] \quad (32)$$

For the Q-learning network (Mnih et al. 2015), randomly sample a batch of data from the DRL memory M_{RL} and update the action-value network Q 's parameters θ_Q using reinforcement learning to minimize the error:

$$L(\theta_Q) = \mathbb{E}_{(s,a,r,s') \sim M_{RL}} \left[\left(r + \gamma \max_{a'} Q(s', a' | \theta'_Q) - Q(s, a | \theta_Q) \right)^2 \right] \quad (33)$$

periodically updates the target network parameters $\theta'_Q \leftarrow \theta_Q$.

4.2.2 ACFP

Perolat et al. (2018) introduced actor-critic fictitious play (ACFP), an actor-critic-based approach tailored for multistage games. ACFP employs a decentralized, online learning algorithm that converges to Nash equilibria through strategy and value function updates

on different timescales. Designed for multistage games, it integrates policy evaluation and control techniques for robust learning.

Both ACFP and NFSP/FSP strive to learn Nash equilibria in multi-agent settings, grounded in game theory and leveraging RL techniques. They aim for decentralized learning, enabling agents to independently optimize strategies without centralized coordination. Both methods utilize iterative processes where agents update strategies based on historical data, incorporating RL to adapt to evolving opponent strategies. However, ACFP distinguishes itself from NFSP through its use of a two-timescale stochastic approximation within an actor-critic framework, incorporating both on-policy and off-policy updates for the critic. While ACFP is tailored for multistage games, NFSP/FSP can be applied to a broader range of game types. The primary differences between ACFP and NFSP lie in action selection, critic updates, and actor updates.

Action selection: In ACFP, each agent selects an action based on its current strategy π_n^i , which is a blend of the current strategy and a best response derived from the critic. This process ensures the strategy evolves by integrating feedback from the critic. Unlike NFSP, which employs a combination of best response and average policies governed by an ϵ -greedy strategy for exploration and exploitation, ACFP directly updates the strategy using a best response function, thereby omitting an explicit exploration-exploitation trade-off. Each agent i selects an action $a_n^i \sim \pi_n^i(\cdot | s_n)$, and the strategy π_n^i is updated as:

$$\pi_{n+1}^i(s_n, \cdot) = (1 - \beta_n)\pi_n^i(s_n, \cdot) + \beta_n B_\sigma(Q_n^i(s_n, \cdot)) \quad (34)$$

where $B_\sigma(Q_n^i(s_n, \cdot))$ is the best response derived from the critic Q_n^i .

Critic update: In ACFP, the critic is updated through either an off-policy control step or a policy evaluation step, involving precise updates to the Q-values based on observed rewards and projected future values. This two-timescale approach ensures that the critic adapts swiftly while the actor updates more gradually. Unlike NFSP, which updates the Q-network via experience replay and Q-learning, ACFP employs a dynamic adjustment process that keeps the critic in alignment with the evolving strategies of the actors. An off-policy control step can be performed either:

$$Q_{n+1}^i(s_n, a_n^i) = (1 - \alpha_n)Q_n^i(s_n, a_n^i) + \alpha_n (r_n^i + \gamma C_\sigma(Q_n^i(s_{n+1}, \cdot))) \quad (35)$$

where $C_\sigma(Q_n^i(s_{n+1}, \cdot)) = \mathbb{E}_{a \sim \pi_{n+1}^i} [Q_n^i(s_{n+1}, a)]$. Or a policy evaluation step:

$$Q_{n+1}^i(s_n, a_n^i) = (1 - \alpha_n)Q_n^i(s_n, a_n^i) + \alpha_n (r_n^i + \gamma \mathbb{E}_{b \sim \pi_n^i(\cdot | s_{n+1})} [Q_n^i(s_{n+1}, b)]) \quad (36)$$

Actor update: In ACFP, the actor update modifies the policy by incorporating the critic's evaluation and best response dynamics, ensuring the policy gradually aligns with optimal actions. This method directly integrates the best response derived from the critic into the existing strategy, without a distinct imitation step, as seen in NFSP. In contrast to NFSP, which uses supervised learning to train an average policy network to imitate the best response policy, ACFP continuously refines its policy through direct feedback from the critic's assessments. The actor step updates the policy using the critic's evaluation and best response dynamics:

$$\pi_{n+1}^i(s_n, \cdot) = (1 - \beta_n)\pi_n^i(s_n, \cdot) + \beta_n B_\sigma(Q_n^i(s_n, \cdot)) \quad (37)$$

where β_n is the learning rate for the actor update, and $B_\sigma(Q_n^i(s_n, \cdot))$ represents the best response derived from the Q-values.

4.2.3 FPEM

Qin et al. (2019) introduced FP with expanding models (FPEM), which leverages a hierarchical policy structure integrating multiple sub-models (base policies) with a policy selector network. In each iteration, FPEM trains a new base policy and adds it to the existing set, while updating the policy selector to dynamically choose among these policies during gameplay. This approach stabilizes the learning process by fixing previously trained policies, mitigating catastrophic forgetting. By maintaining a set of fixed base policies and incrementally adding new ones, FPEM avoids the destabilization of learned strategies that occurs with single model updates.

FPEM is particularly effective in environments with dynamic and diverse opponents. It maintains adaptability and stability in multi-agent interactions by continuously incorporating new base policies while preserving the performance of older ones. This enables a more granular and context-specific decision-making process, as the policy selector adaptively chooses the most suitable base policy based on the current state and opponent behavior. The structured approach of FPEM ensures it can handle a wide range of scenarios without overfitting to specific opponents, leading to improved overall performance and resilience.

A notable innovation of FPEM is the use of a behavior strategy instead of a mixed strategy, providing robust performance in finite simulations. The policy selector network dynamically chooses the appropriate base policy during gameplay, based on the current state and opponent actions, enabling effective responses to environmental and opponent strategy changes. Additionally, reservoir sampling for experience storage ensures a diverse and representative set of experiences for the policy selector, further enhancing robustness and adaptability. Consequently, FPEM demonstrates significant advancements in stability and learning efficiency in multi-agent environments.

Specially, the behavior strategy in FPEM is implemented by the policy selector W , which chooses a base policy at every action history h . The behavior strategy is $\pi_i(h_t, a_t) = \pi_i(s_t, a_t)$, the policy π_i depends on the state s_t and action a_t , ensuring the selection of a base policy at each history step. The base policy π_{i_t} is trained to be a best response to the current opponent's strategy, $\theta_{i_t} = \arg \max_{\theta_i} V(\pi_{i_t}, \pi_{-i}), i(h)$, here $V^{\pi, i}(s) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_t) \mid s_0 = s]$. This objective ensures that each new base policy improves the overall strategy by being a best response to the opponent. For the policy selector W network training, it aims to optimize the following loss function using stored experience pairs (h, j) :

$$\omega_t = \arg \min_{\omega} \mathbb{E}_{(h, j) \sim M} [-\log W(h, j \mid \omega)] \quad (38)$$

it ensures that the policy selector can effectively choose the appropriate base policy at each state.

4.2.4 MCNFSP & ANFSP

Zhang et al. (2021) proposed a new NFSP algorithm that combines Monte Carlo tree search with NFSP, called MC-NFSP. Compared to NFSP, MCNFSP improves by using Monte Carlo Tree Search (MCTS, Coulom (2006)) instead of Deep Q-Network (DQN, Mnih et al. (2015)). NFSP relies on DQN to approximate the best response strategy, but DQN can be unstable in multi-agent non-stationary environments, leading to slow training and potential failure to converge. MCNFSP uses MCTS for search, which has better convergence properties and can explore a wider search space. This allows MCNFSP to more effectively approximate Nash Equilibrium in games with large-scale search spaces and depths.

Furthermore, Zhang et al. (2021) proposed ANFSP (Asynchronous NFSP), which improves upon both NFSP and MCNFSP by employing a parallel, asynchronous architecture. This design allows multiple threads to explore different policies and share gradients in real-time, significantly speeding up the training process and improving scalability. ANFSP reduces the convergence time and memory costs, achieving more stable and efficient learning, particularly in complex IIGs.

4.2.5 LRM-FP

He et al. proposed a novel algorithm called local regret minimization based fictitious play (LRM-FP, He et al. (2022)). LRM-FP calculates counterfactual regret for local state-action values to derive behavioral strategies, showcasing superior performance in large-scale imperfect information games. By integrating this method with Fictitious Play, the LRM-FP algorithm effectively finds approximate Nash equilibria in zero-sum imperfect information games. It iteratively updates strategies through regret matching processes to progressively approximate the best response. Using the average strategy, rather than the strategy from the previous iteration, to compute regret values significantly enhances the algorithm's stability and convergence speed. LRM-FP balances exploration and exploitation through the regret matching process, increasing the probability of exploring suboptimal actions and resulting in superior stochastic strategies. These innovations allow LRM-FP to significantly improve performance in imperfect information games, especially in terms of convergence speed and strategy optimization, thereby outperforming existing methods.

LRMFP integrates local regret minimization (LRM) and FP to find approximate Nash equilibria in zero-sum IIGs. LRM calculates counterfactual regret for local state-action values to derive behavioral strategies, effectively handling partially observable and non-Markovian environments. FP is incorporated by iteratively updating the policy based on regret matching, using average strategies derived from past best responses for enhanced stability and faster convergence. Anticipatory dynamics balance exploration and exploitation, ensuring the strategy approximates the best response, leading to superior performance in imperfect information games. Instead of using the last iteration's strategy, LRM-FP uses the average strategy to calculate the local regret:

$$R_T^\beta(I, a) = Q^\beta(I, a) - \sum_{a' \in A(I)} \pi_T^\beta(I, a') Q^\beta(I, a') \quad (39)$$

where the policy is updated using the RM: $\pi_{T+1}^\beta(I, a) = \frac{[R_T^\beta(I, a)]_+}{\sum_{a' \in A(I)} [R_T^\beta(I, a')]_+}$,

$[x]_+ = \max(x, 0)$. The cumulative counterfactual regret $R_T^\beta(I, a)$ is updated iteratively: $R_T^\beta(I, a) = \sum_{t=1}^T (Q_t^\beta(I, a) - V_t^\beta(I))$. $R_T^{\text{imm}}(I) = \max_{a \in A(I)} (Q^\beta(I, a) - V^\beta(I))$, $V^\beta(I) = \sum_{a \in A(I)} \pi^\beta(I, a) Q^\beta(I, a)$, $Q^\beta(I, a) = \mathbb{E}[u(I, a) + \gamma \max_{a'} Q^\beta(I', a')]$.

The anticipatory dynamic parameter η is set to balance exploration and exploitation, $\sigma^\beta(I) = \eta\beta + (1 - \eta)\pi^\beta$.

4.2.6 AdaptFSP

Goldstein and Brown (2022) proposed AdaptFSP, which enhances the traditional FSP algorithm by integrating an adaptive deep reinforcement learning component that continuously adapts to the best response, thereby mitigating overfitting to the meta-strategy. This innovation accelerates convergence to policies with lower exploitability in continuous control adversarial games. The algorithm is theoretically grounded in Generalized Weakened Fictitious Play, ensuring convergence to Nash equilibria, and its effectiveness is empirically validated across multiple domains, including scenarios with partial observability. This makes AdaptFSP a robust and efficient method for learning unexploitable policies in complex environments.

Unlike traditional FSP, which only trains the best response to the opponents meta-strategy, AdaptFSP includes an adaptive component that trains against a combination of the meta-strategy and a regularizer policy. This reduces overfitting to the meta-strategy. The adaptive regularizer policy is computed as the average of the previously learned best responses:

$$\psi_j^p = \frac{1}{j} \sum_{k=1}^{j-1} \beta_k^p \quad (40)$$

The best response β_j^p to a combination of the opponents meta-strategy π_{i-1}^{-p} and the adaptive regularizer policy ψ_j^p is:

$$\beta_j^p = \text{BR} \left(\frac{i-1}{i} \pi_{i-1}^{-p} + \frac{1}{i} \psi_j^p \right) \quad (41)$$

The meta-strategy update incorporates the newly trained best response in a weighted manner, ensuring that the policy progressively incorporates new best responses while retaining past information. After completing the adaptive training, each player's meta strategy can be updated by:

$$\pi_i^p = \frac{i-1}{i} \pi_{i-1}^p + \frac{1}{i} \beta_M^p \quad (42)$$

4.2.7 OFF-FSP

Chen et al. (2024) proposed offline FSP (OFF-FSP), an innovative offline reinforcement learning algorithm. Unlike NFSP, OFF-FSP does not rely on generating samples through online interactions. Instead, it uses a fixed dataset and importance sampling to simulate interactions with different opponents, enabling strategy updates and approximations of Nash equilibrium. Furthermore, OFF-FSP can handle partially covered datasets by employing single-agent offline reinforcement learning algorithms to learn and estimate uncovered state-action pairs, effectively reducing strategy exploitability. This method integrates various single-agent offline RL approaches, such as OFF-FSP-CQL and OFF-FSP-CRR, showcasing its strong compatibility and flexibility. Additionally, OFF-FSP introduces a novel offline self-play framework, excelling in competitive games by approximating Nash equilibrium even with incomplete data coverage. These advancements allow OFF-FSP to significantly outperform existing state-of-the-art baselines, providing an effective and practical solution for offline reinforcement learning in competitive games.

OFF-FSP uses a fixed dataset and employs importance sampling to simulate interactions with different opponents. The weight for importance sampling when changing the opponent's strategy from π_{-i}^b to π_{-i} is calculated as:

$$w(dt) = \frac{x_{-i}^{\pi_{-i}}(s_j)\pi_{-i}(a_j|s_j)}{x_{-i}^{\pi_{-i}^b}(s_j)\pi_{-i}^b(a_j|s_j)} \quad (43)$$

where $dt = (s_i^t, a_i^t, s_i^{t+1})$ is a data tuple, $x_{-i}^{\pi_{-i}}(s_j)$ and $x_{-i}^{\pi_{-i}^b}(s_j)$ are the reach probabilities under the strategies π_{-i} and π_{-i}^b respectively, $\pi_{-i}(a_j|s_j)$ and $\pi_{-i}^b(a_j|s_j)$ are the probabilities of action a_j given state s_j under the strategies π_{-i} and π_{-i}^b respectively.

In vanilla NFSP, the average strategy π is updated online using generated data. This involves updating the policy networks based on the best response calculated during self-play. However, OFF-FSP updates the strategy using a fixed dataset by combining offline RL methods. The average strategy π_k at iteration k is updated based on samples, and the probability $\pi_k(s, a)$ for all state-action pairs in the dataset is maintained. The update follows the realization-equivalence theorem:

$$\pi_i^k(a|s) = (1 - \lambda)\pi_i^{k-1}(a|s) + \lambda\beta_i^k(a|s) \quad (44)$$

where $\lambda = \frac{\alpha_k x_{\beta_i^k}(s)}{(1 - \alpha_k)x_{\pi_{i}^{k-1}}(s) + \alpha_k x_{\beta_i^k}(s)}$, α_k is the parameter for mixing the best response β_i^k and the previous average strategy π_i^{k-1} , β_i^k is the best response to the opponent's strategy π_{-i}^{k-1} . By leveraging these formulas, OFF-FSP effectively simulates interaction and updates strategies using fixed datasets, which is fundamentally different from the online self-play approach of NFSP.

5 Popular agents in two-player zero-sum IIGs

In the field of IIGs, agents have recently achieved significant successes across specific games, outperforming professional players in evaluations and demonstrating remarkable strategic prowess. In two-player zero-sum IIGs, a milestone was reached in 2015 when Cepheus defeated professional humans in Heads-up Limit Texas Hold'em-marking the first time a computer program outperformed professional humans in a game of such complex incomplete information (Bowling et al. 2015). This breakthrough was followed in 2017 by DeepStack (Moravčík et al. 2017) and Libratus (Brown and Sandholm 2018), each securing victories over human professionals in Heads-up No-limit Texas Hold'em. More recently, in 2022, DeepNash (Perolat et al. 2022) further advanced this field by exceeding professional human performance in Stratego. In multi-player IIGs, similar advancements have been made. In 2019, AlphaStar (Vinyals et al. 2019) and OpenAI Five (Berner et al. 2019) surpassed human professionals in StarCraft II and Dota 2, respectively, while Pluribus (Brown and Sandholm 2019b) achieved a skill level beyond that of professional humans in Six-player No-limit Texas Hold'em, and Suphx (Li et al. 2020) made significant progress in Mahjong. In 2021, DouZero (Zha et al. 2021) also achieved victory over professional humans in DouDizhu game.

While these agents have successfully defeated professional humans within their respective games, their strategies remain largely performance-focused and may lack theoretical guarantees of optimal equilibrium. From a theoretical perspective, many strategies implemented by these agents fall short of optimal equilibrium guarantees and cannot validate whether their outputs represent theoretically optimal strategies. Given this work's focus on equilibrium strategies in IIGs, we will therefore concentrate on the methodologies employed by Cepheus (Bowling et al. 2015), Libratus (Brown and Sandholm 2018), and DeepNash (Perolat et al. 2022).

5.1 Cepheus

Cepheus represents a landmark solution in the domain of IIGs, achieving a major breakthrough (Bowling et al. 2015). Its strategy outperforms professional players in heads-up limit Texas Hold'em, reaching a theoretically unbeatable optimal strategy. Cepheus employs a traditional large-scale game-solving approach, abstracting and decomposing the complete game tree into subgames for strategic analysis. It introduces an enhanced CFR+ algorithm (Tammelin 2014), utilizing positive regret matching and efficient distributed storage compression to significantly improve computational efficiency and scalability, enabling the solution of complex poker scenarios within vast information sets and computational resources.

Cepheus integrates large-scale game tree decomposing, algorithmic optimization, and effective distributed storage techniques (Bowling et al. 2015). Texas Hold'em is first modeled as a complete game tree, then decomposed into 110,565 subgames based on public card combinations and betting action history, with each subgame containing several information sets allocated to specific computing nodes for parallel processing. Cepheus employs the improved CFR+ algorithm (Tammelin 2014), where positive regret matching ensures all regret values remain non-negative, thus preventing issues where negative regrets inhibit strategy optimization and allowing for dynamic adjustments based on updated information, even if a strategy previously underperformed. Additionally, fixed-point arithmetic and

compression convert strategies and regret values into integers and compress them, reducing storage requirements from an initial 262 TB to under 17 TB. Each node utilizes disk caching and streaming compression to optimize I/O efficiency, minimizing memory usage and reducing computational latency. During the computation process, 200 nodes run CFR+ algorithm iterations in parallel, with each node writing updated subgame data to disk after each iteration and periodically calculating strategy exploitability to assess its proximity to Nash equilibrium. When exploitability falls below 1 milli-big-blind per game, the strategy is regarded as near-optimal.

5.2 Libratus

Although Cepheus (Bowling et al. 2015) achieved success by defeating professional humans in Heads-up Limit Texas Hold'em, Heads-up No-limit Texas Hold'em presents greater complexity, larger scale, and more significant challenges for strategy solving. In order to address these challenges, Noam from CMU conducted extensive research and developed the Libratus (Brown and Sandholm 2018) agent based on CFR methods, achieving a substantial advantage over professional players in public evaluations. Unlike previous large-scale game-solving methods, Libratus not only employs abstraction techniques and CFR but also features self-improvement capabilities. The strategy that Libratus generates by solving the abstract game using CFR serves as a baseline, which it then dynamically adjusts during play, enhancing both strategic accuracy and the agent's ability to exploit weaknesses in opponent strategies.

The Libratus (Brown and Sandholm 2018) framework comprises several core modules, enabling dynamic strategy generation, real-time adjustment, and opponent analysis in IIGs. First, Libratus abstracts the full game tree of Texas Hold'em and uses an enhanced CFR algorithm to solve this abstraction, producing an initial strategy library. During gameplay, Libratus dynamically fine-tunes this strategy based on specific game conditions, leveraging real-time analysis to identify and exploit opponent weaknesses and recalculating the game tree at critical decision points to improve precision. Additionally, Libratus decomposes complex subgames, concentrating computational resources to solve optimal strategies in specific scenarios, enabling dynamic optimization within the no-limit betting environment. In opponent analysis, Libratus does not directly construct an opponent model but instead reviews post-game interaction data to refine its own strategic weaknesses, achieving "self-improvement" and enhancing performance in future matches.

5.3 DeepNash

DeepNash innovatively addresses the significant challenges of Stratego, opening new avenues for AI applications in similarly complex IIGs (Perolat et al. 2022). Compared to PIGs like Go and chess, Stratego's state space complexity is extremely high, reaching approximately 10^{535} possible states. Additionally, the challenge of handling private information in Stratego is even more daunting, as the initial deployment phase introduces an enormous hidden space, far surpassing typical IIGs such as poker. In Texas Hold'em poker, players have around 10^3 possible private combinations for a given hand, whereas the initial phase of Stratego involves up to 10^{66} private combinations, resulting in exponentially increased complexity. Moreover, many successful AI algorithms in poker games rely on search and

decomposition techniques that leverage state information, but in Stratego, the depth of hidden information and the vast number of possible states make these techniques impractical. To overcome these challenges, DeepNash introduces a search-free self-play strategy, employing the R-NaD (Regularized Nash dynamics) algorithm to guide AI in a high-dimensional strategy space toward Nash equilibrium (Hennes et al. 2020). R-NaD avoids the complexity of opponent modeling, making strategy learning more direct and efficient, which enables robust performance in complex zero-sum games and allows DeepNash to achieve professional human level play.

DeepNash relies on a meticulously designed deep neural network architecture that combines a U-Net structure with residual blocks and skip connections, featuring four network heads that output value functions and action probability distributions (Perolat et al. 2022). The observation input encoding includes information on both DeepNash's and the opponent's pieces, historical moves, and public information, creating a comprehensive game state representation. Its strategy learning process comprises three key stages: first, reward transformation is conducted to steer the strategy toward Nash equilibrium; then, in the dynamics stage, replicator dynamics are used to reinforce the probability of high-fitness actions, leading to convergence; finally, in the update stage, the fixed-point strategy is used as the regularization strategy for the next iteration, allowing continuous approximation of the original game's Nash equilibrium. To minimize low-probability errors, the model further fine-tunes the strategy in the test phase by filtering out low-probability actions, enhancing the robustness of the strategy. DeepNash not only achieves professional human level performance in the complex version of Stratego for the first time but also offers a general reinforcement learning framework suited for imperfect information decision-making, demonstrating broad application potential.

6 Evaluation experiments

6.1 Testing platform

OpenSpiel is a versatile research platform for reinforcement learning and planning in diverse n-player game settings, including zero-sum, cooperative, general-sum, turn-based, simultaneous-move, perfect and imperfect information games, grid worlds, and social dilemmas (Lanctot et al. 2019). Our study leverages this platform for experimental analysis.

Poker games, particularly Texas Hold'em and its simplified variants Kuhn (Kuhn 1950) and Leduc (Southey et al. 2005) poker, stand as a pivotal example in the field of IIGs due to its hidden information, strategic deception, and complexity. These games serve as benchmarks for evaluating equilibrium strategy-solving algorithms, with notable successes achieved through applications of CFR and other methods (Bowling et al. 2015; Brown and Sandholm 2018; Moravčík et al. 2017). Consequently, poker has garnered extensive research attention and resources as a crucial testbed for assessing strategies in incomplete information settings.

In poker, actions encompass betting, calling, raising, checking, and folding. Kuhn Poker, the simplest variant, involves only three cards per player, with no community cards and a single round of betting. Cards rank as "King" > "Queen" > "Jack," determining the winner at showdown. Leduc Poker adds complexity, featuring six cards, two rounds of betting, and

a community card in the second round. Player 1 acts first in both rounds, with a maximum of two raises allowed in the first round. The player with matching private and community cards wins at showdown; otherwise, the higher card prevails.

6.2 Evaluation metrics

In solving IIGs, the objective is to find a Nash equilibrium strategy or an approximate one. While equilibrium-solving algorithms should theoretically yield a Nash equilibrium, in practice, computing an approximate Nash equilibrium is often sufficient. We adopt a widely accepted evaluation method, similar to that used in other research, which involves measuring the exploitability of a strategy to assess algorithm effectiveness. Exploitability, a classic evaluation metric, measures the effectiveness of the strategy derived by the game-solving algorithm. Lower exploitability indicates that the strategy is closer to Nash equilibrium, representing a superior game strategy.

Specifically, in two-player zero-sum games, the exploitability $e(\sigma_i)$ of a strategy σ_i reflects its deviation from the best response (Johanson et al. 2007). When the exploitability of a strategy is zero, it signifies that the strategy has reached a Nash equilibrium and cannot be beaten by any other strategy. In an ϵ -Nash equilibrium, the exploitability of any player's strategy does not exceed ϵ , ensuring the stability and quality of the game strategy. The exploitability $e(\sigma_i)$ of a strategy can be defined as:

$$e(\sigma_i) = u_i(\sigma_i^*, BR(\sigma_i^*)) - u_i(\sigma_i, BR(\sigma_i)) \quad (45)$$

where $e(\sigma_i)$ represents the exploitability, u_i is the utility function for player i , σ_i^* is the Nash equilibrium strategy for player i , $BR(\sigma_i)$ denotes the best response to strategy σ_i .

6.3 Experimental setting

Six types of games are used to evaluate equilibrium-solving algorithms, including Kuhn poker, Leduc poker, Liar's dice, and three variants of Leduc poker. Leduc8 and Leduc10 denote Leduc poker variants containing 8 and 10 cards, respectively. Both of these Leduc variants are larger in scale than the vanilla 6-card Leduc poker, yet their game rules remain consistent. The 3-player Leduc variant extends the vanilla 2-player Leduc poker to three players, simply increasing the number of players while keeping the game rules and card count consistent with the original Leduc poker; that is, there are still a total of two rounds and six cards in play. Liar's Dice (Ferguson 1970) here is also a two-player game. It is a bluffing game in which players attempt to deceive each other about the dice they hold. Each player rolls a set of dice in secret, then takes turns making bids on the total number of dice showing a certain face, with the option to call a bluff if they believe a bid is overstated.

Our primary aim here is to better illustrate how the utility of equilibrium strategy algorithm evaluation metrics can be quantified. We selected multiple classic algorithms from the CFR and FP series for demonstration, including CFR (Zinkevich et al. 2007), CFR+ (Tamelin 2014), CFR-BR (Johanson et al. 2011), DCFR (Brown and Sandholm 2019a), LCFR (Brown and Sandholm 2019a), OS-MCCFR (Gibson et al. 2012a), ES-MCCFR (Gibson et al. 2012a), DeepCFR (Brown et al. 2019), XFP (Heinrich et al. 2015), and NFSP (Heinrich and Silver 2016). These algorithms are based on open-source code from the OpenSpiel

platform (Lanctot et al. 2019), with the exception of the LCFR algorithm, which we reproduced. In two-player games, we use exploitability as the standard evaluation metric, while in three-player zero-sum games, we adopt NashConv as the evaluation metric. Although Nash equilibrium strategies lack theoretical guarantees in multi-player settings, they still offer strong reference value. The purpose of our experiments is simply to further explore the applicability of these algorithms in multi-player scenarios. For the number of iterations, we set values between 1,000 and 10,000 based on parameters of previous classic algorithms and the specifics of each game. For DeepCFR, we set the iteration count uniformly to 1,000, as this is sufficient to demonstrate the algorithm's performance.

6.4 Experimental results

To present the experimental results more clearly, we provide detailed visualization of curve changes within the early iteration range in each figure. The X-axis represents the iteration number, while the Y-axis denotes exploitability, where lower exploitability signifies better algorithm performance, indicating that the strategy is closer to a Nash equilibrium. When exploitability reaches zero, the strategy is considered to have achieved Nash equilibrium. It is important to note that, as CFR-BR is applicable only to two-player games, it was not included in the three-player Leduc experiment.

As shown in Figs. 8, 9, 10 and 11, it is evident that all algorithms display a convergence trend. As the number of iterations increases, the curves progressively approach zero and stabilize. These results firstly verify the effectiveness of the algorithms in solving equilibrium or approximate equilibrium strategies and affirm that our selection of multiple evaluation metrics is objective. Furthermore, the eventual flattening of each algorithm's curve suggests that the iteration counts we specified are reasonable and sufficiently reflect each algorithm's performance (Fig. 9, 10).

As shown in Figs. 8, 9, 10, 11 and 12, it can be found that the OS-MCCFR and ES-MCCFR algorithms generally underperform relative to other algorithms, particularly in Fig. 12, where OS-MCCFR, although stable, shows limited convergence and weaker strategy performance. This is largely because the CFR series algorithms have an advantage in strategy computation for poker games, while Liar's Dice does not fall into this category. Additionally, OS-MCCFR and ES-MCCFR are MCCFR variants that employ node sampling within the game tree for iterative solving. Although these approaches broaden the CFR

Fig. 8 Results on Kuhn poker game

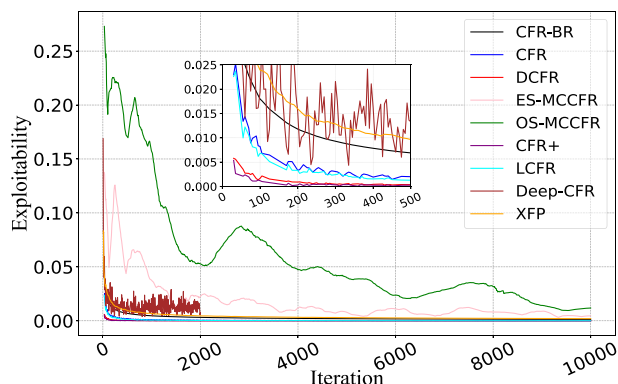


Fig. 9 Results on Leduc poker game

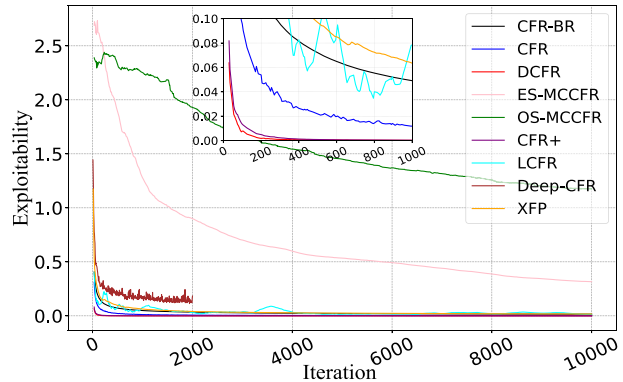


Fig. 10 Results on Leduc8 poker game

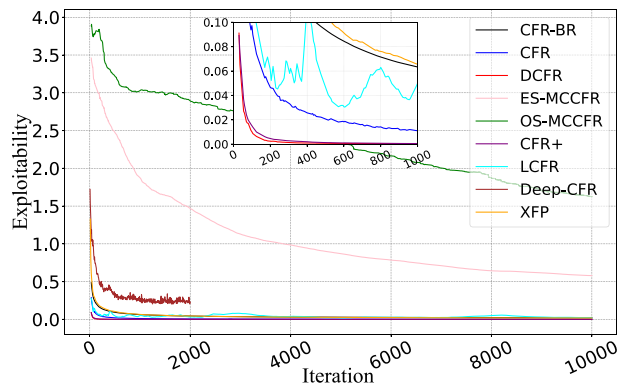
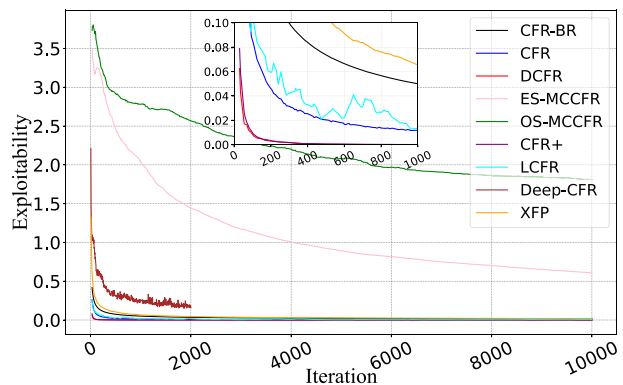


Fig. 11 Results on Leduc10 poker game



algorithm's solution scope, their performance is inherently limited compared to CFR's full traversal of the game tree.

Figures 8, 9, 10, 11 and 12 also reveal that, while DCFR slightly underperforms CFR+ during early iterations in Kuhn poker, it demonstrates superior performance in other games, indicating the effectiveness of DCFR's regret discounting approach. The DeepCFR algorithm shows a convergence trend, yet its effectiveness is markedly lower than that of mul-

tuple CFR algorithms, and its curve exhibits greater fluctuation. This is because DeepCFR combines neural networks to train both value and policy networks for equilibrium strategy computation. As the first end-to-end equilibrium strategy algorithm within the CFR series, DeepCFR shows particular advantages in solving ultra-large-scale game strategies.

Figures 8, 9, 10, 11 and 12 clearly illustrate the convergence trends and rates of various comparison methods. However, in order to better illustrate the changes of various CFR methods during the iterative process, we have further provided the exploitability under certain numbers of iterations here, so as to present it from a numerical perspective. The highlighted numbers represent the lowest values in the corresponding iterations, and the single underlined and double underlined parts represent the second and third lowest values, respectively. In addition, given that prior studies (e.g., Libratus (Brown and Sandholm 2018)) typically limit the number of CFR iterations to 1500 or fewer, we have included more granular data for the first 1000 iterations in Table.3 to facilitate detailed analysis.

For XFP, Figs. 8, 9, 10, 11 and 12 show a trend of convergence, and the exploitability continuously decreases as the number of iterations increases. Meanwhile, compared with CFR, CFR-BR, DCFR and CFR+, it converges more slowly and has a higher exploitability under the same number of iterations. However, it also converges faster than MCCFR series, and at the same time, it is also superior to the DeepCFR. For NFSP, we adopt the evaluation method from the paper of DeepCFR (Brown et al. 2019). The X-axis represents the number of nodes touched, and the Y-axis represents the exploitability. As shown in Figs. 14, 15, 16 and 17, it can be found that NFSP shows a gradually converging trend across four testing games. Compared to DeepCFR, NFSP converges more slowly. Among the four testing games, we also found that as the game scale increased, the final performance of NFSP gradually approached that of DeepCFR (Fig. 15, 16).

Figure 13 illustrates that these algorithms exhibit a gradual convergence trend in multi-player settings as well. Although CFR series algorithms have robust theoretical support for Nash equilibrium solutions in two-player games, this does not preclude their application to multi-player games. Rather, the strategies derived in multi-player settings currently lack complete theoretical backing. Nonetheless, these derived strategies still provide valuable decision-making insights. For instance, the CFR-based agent Pluribus (Brown and Sandholm 2019b) has achieved success against professional human players in six-player Texas Hold'em, further suggesting a potential direction for future research: even in the absence of full theoretical guarantees, successful equilibrium-solving algorithms from two-player games may still prove effective for multi-player strategy computation.

Fig. 12 Results on Liar's dice

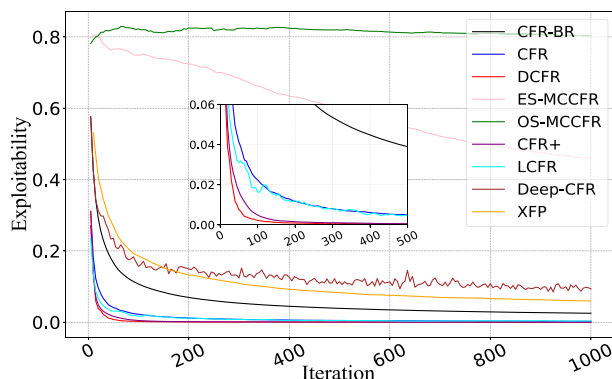


Table 3 Exploitability of CFR comparison methods in different iterations

Method	Iteration								
	30	100	200	300	400	600	1000	2000	3000
<i>Kuhn Poker game</i>									
CFR	0.0229	0.0082	<u>0.0033</u>	0.0030	0.0019	0.0013	0.0010	0.0005	0.0004
CFR-BR	0.0399	0.0179	0.0118	0.0092	0.0078	0.0062	0.0046	0.0032	0.0026
OS-MCCFR	0.2728	0.1914	0.2255	0.2204	0.1986	0.1943	0.1553	0.0525	0.0835
ES-MCCFR	0.1375	0.0844	0.0962	0.1048	0.0676	0.0653	0.0367	0.0233	0.0201
CFR+	0.0053	0.0012	0.0003	0.00028	0.0002	0.00017	0.00015	0.00007	0.00003
LCFR	<u>0.0226</u>	<u>0.00681</u>	0.00332	<u>0.00249</u>	<u>0.00155</u>	<u>0.00117</u>	<u>0.00064</u>	<u>0.00032</u>	<u>0.00023</u>
DCFR	<u>0.0058</u>	<u>0.0018</u>	<u>0.00091</u>	<u>0.00045</u>	<u>0.00044</u>	<u>0.00035</u>	<u>0.00017</u>	<u>0.00011</u>	<u>0.00006</u>
<i>Leduc Poker game</i>									
CFR	<u>0.3109</u>	<u>0.0957</u>	<u>0.0538</u>	<u>0.0346</u>	<u>0.0264</u>	<u>0.0198</u>	<u>0.0119</u>	<u>0.0064</u>	<u>0.0049</u>
CFR-BR	0.4062	0.1927	0.1231	0.0947	0.0814	0.0649	0.0487	0.0344	0.0282
OS-MCCFR	2.3892	2.3317	2.3872	2.4164	2.4142	2.3	2.2526	1.9268	1.6897
ES-MCCFR	2.6887	2.6816	2.4942	2.1896	2.039	1.703	1.2648	0.8973	0.7015
CFR+	<u>0.0817</u>	<u>0.0134</u>	<u>0.0049</u>	<u>0.0022</u>	<u>0.0012</u>	<u>0.0006</u>	<u>0.0003</u>	<u>0.00009</u>	<u>0.00004</u>
LCFR	0.4048	0.1329	0.1693	0.14	0.0826	0.0885	0.0792	0.0291	0.0375
DCFR	0.0638	0.0077	0.0017	0.0008	0.0007	0.0005	0.00015	0.00003	0.00002
<i>Leduc8 Poker game</i>									
CFR	0.2879	<u>0.0973</u>	<u>0.0466</u>	<u>0.0332</u>	<u>0.0257</u>	<u>0.01831</u>	<u>0.01109</u>	<u>0.00601</u>	<u>0.0047</u>
CFR-BR	0.4867	0.2386	0.1581	0.1238	0.10643	0.08444	0.06344	0.04359	0.0359
OS-MCCFR	3.8864	3.7788	3.791	3.3947	3.31039	3.09712	3.02627	2.88581	2.75028
ES-MCCFR	3.4595	3.1275	2.9252	2.7258	2.58351	2.36981	1.87867	1.46036	1.12804
CFR+	0.0891	<u>0.0151</u>	<u>0.0046</u>	<u>0.0027</u>	<u>0.00198</u>	<u>0.00102</u>	<u>0.00049</u>	<u>0.00023</u>	<u>0.00013</u>
LCFR	<u>0.2773</u>	0.1339	0.0609	0.0581	0.11535	0.03064	0.04815	0.03131	0.07811
DCFR	<u>0.0911</u>	0.0101	0.0025	0.0013	0.00096	0.00061	0.00031	0.00008	0.00003
<i>Leduc10 Poker game</i>									
CFR	0.26	<u>0.0857</u>	<u>0.0461</u>	<u>0.0312</u>	<u>0.0255</u>	<u>0.0168</u>	<u>0.0118</u>	<u>0.0064</u>	<u>0.0045</u>
CFR-BR	0.419	0.1955	0.1274	0.0987	0.0839	0.0665	0.0501	0.0343	0.0277
OS-MCCFR	3.732	3.7009	3.2828	3.0922	2.9645	2.8596	2.7816	2.5634	2.3321
ES-MCCFR	3.494	3.2107	3.1687	2.8359	2.6388	2.3327	2.0423	1.4399	1.1736
CFR+	<u>0.079</u>	<u>0.0098</u>	0.0031	0.0016	<u>0.0009</u>	<u>0.0004</u>	<u>0.00016</u>	<u>0.00045</u>	<u>0.00002</u>
LCFR	<u>0.252</u>	0.1121	0.0594	0.0424	0.0329	0.0226	0.0134	0.0176	0.0055
DCFR	0.063	0.0086	<u>0.0034</u>	<u>0.0019</u>	0.0008	0.0003	0.00009	0.00002	0.00001
<i>Liar's dice game</i>									
	30	100	200	300	400	500	600	800	1000
CFR	0.0667	0.0225	<u>0.0119</u>	0.0081	0.0062	0.0050	0.0042	<u>0.0032</u>	<u>0.0027</u>
CFR-BR	0.2306	0.1071	0.0694	0.0535	0.0448	0.0390	0.0347	0.0289	0.0252
OS-MCCFR	0.8130	0.8223	0.8251	0.8241	0.8245	0.8191	0.8136	0.8100	0.8043
ES-MCCFR	0.7796	0.7488	0.7244	0.6852	0.6338	0.5962	0.5552	0.4964	0.4599
CFR+	<u>0.0308</u>	0.0049	<u>0.0015</u>	<u>0.0010</u>	<u>0.00078</u>	<u>0.00051</u>	<u>0.00037</u>	<u>0.00020</u>	<u>0.00013</u>
LCFR	<u>0.0470</u>	<u>0.0163</u>	0.0121	<u>0.0079</u>	<u>0.0055</u>	<u>0.0049</u>	<u>0.0039</u>	0.0038	0.0034
DCFR	0.0222	<u>0.0224</u>	0.00094	0.00096	0.00048	0.00045	0.00028	0.00008	0.00009

7 Existing problems and future work

Generalizability of equilibrium strategy solution methods in two-player IIGs. Current equilibrium strategy solution methods for two-player zero-sum IIGs, such as the CFR series and

Fig. 14 Convergence of Deep-CFR and NFSP on Kuhn poker game

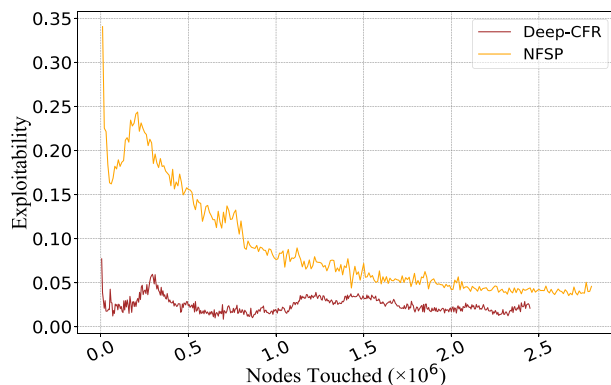


Fig. 15 Convergence of Deep-CFR and NFSP on Leduc poker game

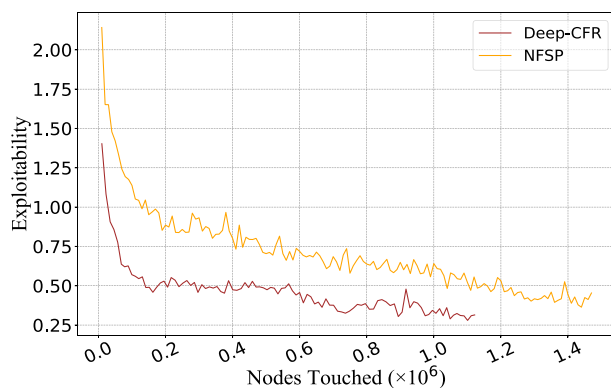
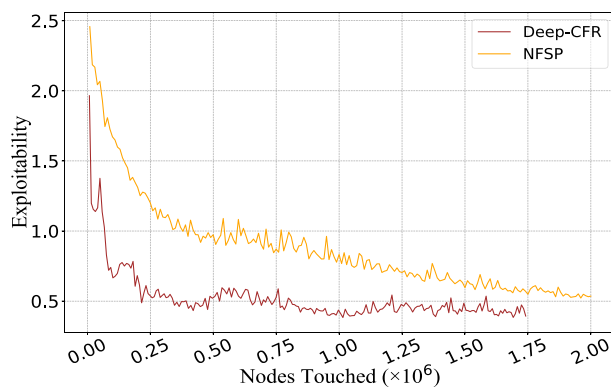


Fig. 16 Convergence of Deep-CFR and NFSP on Leduc8 poker game



the NFSP series, which achieved great success in specific environments like poker games. They have trained agents capable of matching or surpassing professional players through self-play and strategy updates. Theoretically, these methods guarantee strategy optimal-

Fig. 17 Convergence of Deep-CFR and NFSP on Leduc10 poker game

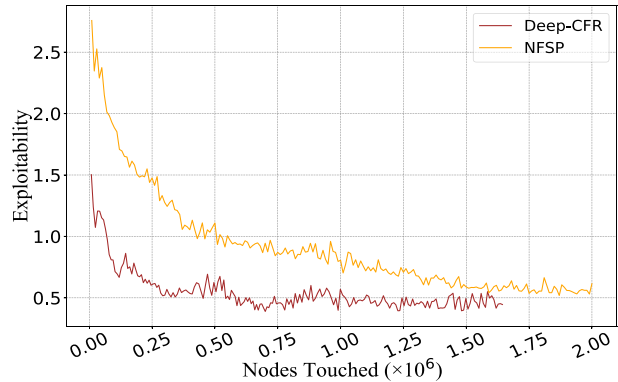
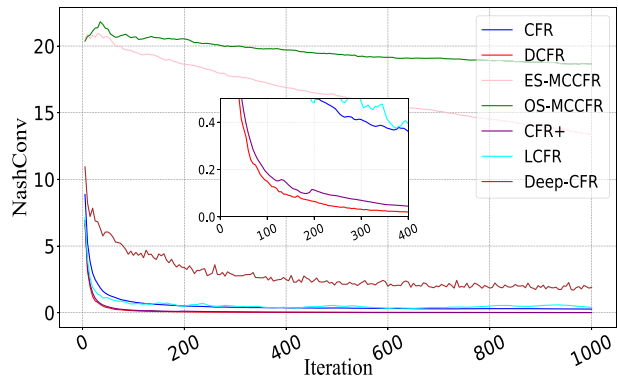


Fig. 13 Results on 3-player Leduc poker game



ity, providing a foundation for the development of AI in games. However, their success is largely confined to poker games, highlighting a generalizability bottleneck. Poker games possess structured characteristics such as limited action spaces and low complexity per round. This allows CFR and NFSP to achieve good results with limited sampling and simpler optimization. In more complex or structurally different games, their adaptability and efficiency decrease significantly, making them difficult to apply effectively.

Possible solutions. To improve the generalizability of solution methods for two-player IIGs, further research should focus on their adaptability across games with diverse structures and information characteristics. On one hand, we can study how to transfer the successful elements of methods like CFR and NFSP to other types of games, enhancing their efficiency in complex games through algorithm refining or new optimization strategies. On the other hand, we can explore novel game modeling methods to better capture essential features of different games, providing more accurate input for solution methods. Additionally, leveraging machine learning and deep learning to automatically learn and optimize strategies could further boost the generalizability and practicality of these methods.

Opponent strategy utilization in equilibrium solution methods for two-player IIGs. Existing equilibrium solution methods for two-player IIGs typically rely on assumptions

of rationality or bounded rationality of players when computing Nash or approximate Nash equilibria. This enables strategies that break even in expected returns, preventing long-term exploitation. However, real-world players often exhibit non-rational behavior with higher reward expectations. Consequently, pursuing theoretically unexploitable strategies may fail practical requirements, particularly when players pursue additional gains. This limitation reduces method effectiveness in actual games and hinders their potential value realization.

Possible solutions. To enhance the application of AI in games, introducing opponent modeling and adaptive exploitation capabilities is essential. This can be achieved by developing opponent-adaptive learning algorithms that identify strategic weaknesses and adjust tactics for increased exploitation. Furthermore, exploring deep learning techniques to automatically learn and predict opponent behavior patterns would enable more precise counter-strategies. Future research should investigate integrating opponent exploitation with equilibrium solution methods, enhancing rewards while maintaining strategic equilibrium.

Refinement of equilibrium theory and efficient strategy solving for multi-player IIGs. This survey reviews Nash equilibrium solution methods for two-player zero-sum IIGs. However, Nash equilibria provides no optimality guarantees in multi-player IIGs and faces equilibrium selection challenges (Daskalakis et al. 2009). Multi-player equilibrium research primarily focuses on maximin equilibria in adversarial team scenarios (Von Stengel and Koller 1997), employing methods like column generation, game tree decomposition, and linear programming (Hansen et al. 2008; Basilico et al. 2017b, a; Zhang and An 2020; Zhang et al. 2022, 2023). Significant theoretical challenges persist in multi-player equilibrium solving. Most existing frameworks rely on restrictive assumptions, such as shared utility functions within teams, which is rarely satisfied in practice. Computational efficiency and scalability present further limitations: traditional optimization methods struggle with large-scale multi-player games, demanding immense computational resources. These constraints hinder practical application of existing methods in multi-player problems.

Possible solutions. To address multi-player game strategy challenges, we must integrate innovative algorithms to improve efficiency and reduce computational demands. Firstly, multi-agent reinforcement learning (MARL, Yang and Wang (2020)) can derive strategies better suited to multi-player dynamics by simulating agent interactions in complex environments. Research should explore integrating MARL with existing equilibrium methods to leverage its strengths in handling complex interactions. Furthermore, distributed computing (Hajibaba and Gorgin 2014) accelerates solution processes and reduces single-node burdens. Developing efficient distributed architectures for large-scale games is also critical. Concurrently, optimizing data communication and synchronization mechanisms remains essential to minimize latency and overhead.

Beyond these approaches, we should explore emerging algorithms like heuristic search and population-based training (PBT) (Li et al. 2019; Dushatskiy et al. 2023) to enhance multi-player game strategy solving. Concurrently, interdisciplinary integration promotes innovations in this domain. Combining game theory with economics, sociology, and psychology provides broader perspectives for examining multi-player game dynamics, yielding stronger theoretical foundations for solution methods.

8 Conclusion

In recent years, equilibrium strategy-solving methods for two-player and multi-player IIGs have advanced significantly, surpassing human professional performance in multiple public evaluations for specific games. To our knowledge, we presents the first comprehensive overview specifically tailored to the domain of two-player zero-sum IIGs, focusing on Nash equilibrium-solving methods. It covers foundational aspects of game model definitions and solution paradigms, as well as detailed discussions of classic solving techniques CFR and FSP. The survey not only presents the original formulations but also analyzes recent improvements that have enhanced their efficacy and robustness.

In the experimental section, the objective comparative analysis of several classic methods is conducted on a public platform. This analysis evaluates their performance across various games, providing insights into their strengths and weaknesses under different conditions. To facilitate future advancements, we propose several directions and suggestions aimed at enhancing the applicability, scalability, and practical effectiveness of these solving methods.

CFR calculation for 'Rock-Paper-Scissors' game

In this section, we provide a detailed calculation process for each variable involved in the CFR through the 'Rock-Paper-Scissors' game. This game is a two-player zero-sum game, and both player can take three actions of 'Rock', 'Paper' and 'Scissor'. Considering that we are here merely to facilitate readers' easier understanding of the various variables and calculation procedures in CFR, we demonstrate only the first iteration's computation process.

Initial settings

For the 'Rock-Paper-Scissors' game, we firstly set the initial random strategy for both players are $\sigma^0 = (a_r : 0.4, a_p : 0.35, a_s : 0.25)$, where a_r , a_p and a_s represent the actions of 'Rock', 'Paper' and 'Scissor', respectively. Rewards are 1 for a win, 0 for a draw and -1 for a loss. The payoff matrix of the game is shown in the Table.2. Assume that the first actual game is 'rock-paper', and the player 1 gets -1 and player 2 gets 1 after this actual game is over. The initial total regret for both players is 0.

Player 1's CFR calculation ($t = 1$)

In this section we provide the full CFR Calculation for the first iteration using the initial settings A.1. As presented in section 3.1, no average strategy calculation is required in this initial iteration. We therefore focus on computing the total regrets and updating the strategy.

Calculation of player 1's total regret

For calculating total regret of Player 1 (Equation.6), immediate regret $r_{1,1}(I, a)$ (Equation.3), counterfactual value $v_1^{\sigma^0}(I, a)$ (Equation.4) at information set I during the first iteration ($t = 1$) should be calculated firstly.

For the counterfactual value $v_1^{\sigma^0}(I)$:

$$\begin{aligned} v_1^{\sigma^0}(I) &= \sum_{h \in I} (\pi_{-i}^{\sigma}(h) \sum_{z \in Z} (\pi^{\sigma}(h, z) u_i(z))) \\ &= 0.4 \times (0.35 \times 1 \times (-1)) + 0.35 \times (0.35 \times 1 \times 0) + 0.25 \times (0.35 \times 1 \times 1) \\ &= -0.0525 \end{aligned} \quad (A1)$$

For the immediate regret $r_{1,1}(I, a)$:

- for the action a_r : $r_{1,1}(I, a_r) = -0.35 - (-0.0525) = -0.2975$
 - for the action a_p : $r_{1,1}(I, a_p) = 0 - (-0.0525) = 0.0525$
 - for the action a_s : $r_{1,1}(I, a_s) = 0.35 - (-0.0525) = 0.4025$
- For the total regret $R_{1,1}(I, a)$:
- for the action a_r : $R_{1,1}(I, a_r) = \sum_{t=1}^T r_t(I, a) = -0.2975$
 - for the action a_p : $R_{1,1}(I, a_p) = 0 + 0.0525 = 0.0525$
 - for the action a_s : $R_{1,1}(I, a_s) = 0 + 0.4025 = 0.4025$
- We usually set the total regret that is less than 0 to 0. Thus here for the action a_r , its total regret $R_{1,1}(I, a_r) = 0$.

Calculation of Player 1's updating strategy

According the Equation.8, the strategy of next iteration for player 1 is:

$$\begin{aligned} \sigma_1^1(I, a) &= \left(\frac{0}{0.0525 + 0.4025}, \frac{0.0525}{0.0525 + 0.4025}, \frac{0.4025}{0.0525 + 0.4025} \right) \\ &= (0, 0.115, 0.885) \end{aligned} \quad (A2)$$

that is, at iteration $t = 2$, the iterative strategy adopted by Player 1 will be $(a_r : 0, a_p : 0.115, a_s : 0.885)$. Through the above two steps of the calculation process, we gradually calculated the total regret and the updated iterative strategy for player 1. The CFR method converges or meets termination conditions through repeated iterations.

Player 2's CFR calculation ($t = 1$)

Under the initial setting conditions A.1, the calculation method for player 2 is the same as that of player 1. We also provide a detailed calculation process.

Calculation of player 2's total regret

For the counterfactual value $v_2^0(I)$:

$$v_2^0(I) = 0.4 \times (0.4 \times 1 \times 0) + 0.35 \times (0.4 \times 1 \times 1) + 0.25 \times (0.4 \times 1 \times (-1)) \quad (\text{A3})$$

$$= 0.04$$

For the immediate regret $r_{2,1}(I, a)$:

- for the action a_r : $r_{2,1}(I, a_r) = 0 - 0.04 = -0.04$
 - for the action a_p : $r_{2,1}(I, a_p) = 0.4 - 0.04 = 0.36$
 - for the action a_s : $r_{2,1}(I, a_s) = -0.4 - 0.04 = -0.44$
- For the total regret $R_{2,1}(I, a)$:

- for the action a_r : $R_{2,1}(I, a_r) = 0 + (-0.04) = -0.04$
 - for the action a_p : $R_{2,1}(I, a_p) = 0 + 0.36 = 0.36$
 - for the action a_s : $R_{2,1}(I, a_s) = 0 + (-0.44) = -0.44$
- here for the action a_r , its total regret $R_{2,1}(I, a_r) = 0$, and for the action a_s , its total regret $R_{2,1}(I, a_s) = 0$

Calculation of player 2's updating strategy

According the Equation.8, the strategy of next iteration for player 2 is:

$$\sigma_2^1(I, a) = \left(\frac{0}{0.36}, \frac{0.36}{0.36}, \frac{0}{0.36} \right) \quad (\text{A4})$$

$$= (0, 1, 0)$$

that is, at iteration $t = 2$, the iterative strategy adopted by Player 2 will be $(a_r : 0, a_p : 1, a_s : 0)$.

Exploitability of CFR comparison methods in different iterations

The experimental results in Sect. 6 demonstrate the convergence trends of various CFR methods. To provide a more detailed comparison of different CFR algorithms, we present their exploitability at multiple iteration points in Table 4. The lowest and second-lowest values are highlighted and underlined, respectively.

Table 4 Exploitability of CFR comparison methods in different iterations

Method	Iteration													
	35	70	105	170	215	270	310	370	425	475	515	585	650	715
Kuhn Poker game														
CFR	0.02529	0.012016	0.006807	0.004299	0.004473	0.003135	0.002791	0.002354	0.002387	0.00207	0.001977	0.001455	0.000944	0.001471
CFR-BR	0.035899	0.022809	0.017621	0.013118	0.011311	0.009863	0.009118	0.008218	0.00757	0.007095	0.006777	0.006303	0.005926	0.005627
OS-MCCFR	0.266587	0.239465	0.200324	0.21575	0.225381	0.221838	0.220772	0.208266	0.190829	0.172174	0.169665	0.191687	0.206233	0.190064
ES-MCCFR	0.135832	0.104866	0.076935	0.076472	0.105812	0.115484	0.101914	0.077842	0.058148	0.043783	0.055457	0.062754	0.063223	0.064701
CFR+	0.002659	0.001982	0.001019	0.000299	0.000201	0.000485	0.00011	0.00036	0.000249	0.000256	0.000285	0.000167	0.000218	0.000182
LCFR	0.023717	0.010539	0.006744	0.003415	0.003015	0.003074	0.002756	0.002094	0.001619	0.001318	0.001264	0.00122	0.000977	0.00093
DCFR	0.005596	0.002689	0.002165	0.001035	0.000789	0.000473	0.000531	0.000535	0.000498	0.000395	0.000339	0.000239	0.00025	0.000268
Leduc Poker game														
CFR	0.263942	0.136235	0.090335	0.056472	0.046554	0.037356	0.032171	0.02684	0.025146	0.023415	0.020883	0.021465	0.017544	0.016093
CFR-BR	0.371129	0.242887	0.187908	0.135169	0.116897	0.101905	0.093929	0.084864	0.078738	0.074113	0.070772	0.065813	0.06197	0.058758
OS-MCCFR	2.379823	2.358218	2.32187	2.333786	2.421807	2.423486	2.418691	2.408448	2.401741	2.360501	2.303958	2.300606	2.288275	2.272701
ES-MCCFR	2.724319	2.626654	2.622698	2.526558	2.435367	2.270553	2.175816	2.071891	2.017072	1.928978	1.865275	1.710631	1.68822	1.617049
CFR+	0.058423	0.02184	0.012723	0.006143	0.004065	0.003028	0.002128	0.001509	0.001178	0.000919	0.000839	0.000658	0.000554	0.000485
LCFR	0.33636	0.151466	0.133766	0.11421	0.210671	0.200928	0.127713	0.069906	0.069085	0.06449	0.096807	0.080143	0.051598	0.040848
DCFR	0.047604	0.01577	0.007752	0.002941	0.001951	0.001172	0.000924	0.000746	0.000698	0.000654	0.000557	0.000502	0.000399	0.000295
Leduc8 Poker game														
CFR	0.247807	0.12218	0.089886	0.054371	0.045772	0.036193	0.032426	0.027665	0.02466	0.022171	0.019726	0.018476	0.016936	0.014973
CFR-BR	0.447785	0.289963	0.231345	0.174179	0.151407	0.132468	0.122743	0.111297	0.102888	0.096563	0.092152	0.085685	0.080601	0.076334
OS-MCCFR	3.905932	3.741193	3.782913	3.808641	3.710677	3.550028	3.392695	3.322158	3.271653	3.242312	3.208353	3.109742	3.079913	3.074095
ES-MCCFR	3.419744	3.223603	3.107749	2.992278	2.909221	2.816906	2.727723	2.62717	2.553659	2.498883	2.460516	2.381387	2.278272	2.175998
CFR+	0.089145	0.024342	0.01398	0.006132	0.004076	0.003195	0.002708	0.00223	0.001674	0.001369	0.001209	0.001039	0.000901	0.000817
LCFR	0.263942	0.101543	0.114946	0.062927	0.063796	0.05243	0.062156	0.051511	0.096738	0.057394	0.05042	0.031796	0.039152	0.045956
DCFR	0.065461	0.018532	0.008727	0.003137	0.002438	0.001587	0.001264	0.00101	0.00088	0.000806	0.000727	0.000647	0.000538	0.000516
Leduc10 Poker game														
CFR	0.227551	0.117551	0.083252	0.053905	0.042167	0.033973	0.033585	0.027341	0.023675	0.020866	0.020075	0.018051	0.016544	0.014534
CFR-BR	0.37648	0.244936	0.189793	0.140373	0.121713	0.106115	0.097607	0.087803	0.080937	0.075858	0.072453	0.067435	0.063599	0.060226

Table 4 (continued)

Method	Iteration														
	35	70	105	170	215	270	310	370	425	475	515	585	650	715	
OS-MCCFR	3.783875	3.742173	3.62908	3.38439	3.244413	3.15094	3.079713	2.981012	2.948207	2.93811	2.906074	2.872099	2.844022	2.826855	
ES-MCCFR	3.382091	3.195948	3.222525	3.242257	3.138771	2.919077	2.827481	2.684667	2.616407	2.511003	2.435567	2.350569	2.290493	2.238974	
CFR+	0.058984	0.018116	0.008782	0.003569	0.00285	0.001978	0.001497	0.001009	0.000877	0.000649	0.000583	0.000452	0.000384	0.000317	
LCFR	0.22159	0.119642	0.102307	0.072422	0.067031	0.04798	0.042889	0.037291	0.031052	0.020965	0.0265	0.021963	0.041129	0.038	
DCFR	0.042844	0.016292	0.007411	0.004218	0.00281	0.002174	0.00181	0.001025	0.000822	0.000571	0.000414	0.000336	0.000263	0.000178	
Liar's dice game															
	45	85	145	185	245	285	345	385	445	485	545	585	645	685	
CFR	0.045056	0.025635	0.016067	0.012921	0.009596	0.00828	0.00696	0.006565	0.005461	0.005132	0.004486	0.004218	0.003907	0.003826	
CFR-BR	0.179404	0.118185	0.085679	0.073011	0.061059	0.055215	0.049009	0.045822	0.041892	0.039664	0.036878	0.035238	0.033161	0.031906	
OS-MCCFR	0.81836	0.824721	0.812026	0.817622	0.824877	0.819627	0.823805	0.826514	0.820494	0.819587	0.817167	0.815153	0.810689	0.81332	
ES-MCCFR	0.762725	0.754548	0.739111	0.7283	0.709096	0.69235	0.659508	0.639661	0.61946	0.603382	0.575874	0.559194	0.542866	0.528129	
CFR+	0.019197	0.006697	0.002326	0.00171	0.00127	0.001074	0.000917	0.000836	0.000628	0.000536	0.000432	0.000388	0.000325	0.000279	
LCFR	0.032314	0.018465	0.015469	0.012175	0.00956	0.008264	0.007637	0.005421	0.004981	0.004477	0.004491	0.004291	0.003131	0.003243	
DCFR	0.010084	0.002886	0.001278	0.001005	0.000757	0.000597	0.00058	0.00048	0.000487	0.000475	0.000394	0.000308	0.000188	0.000152	

Author contributions Huale Li and Shuhan Qi conceptualized the study, designed the methodology, and wrote the main manuscript text. Huale Li also developed the software, conducted the investigation, and performed validation. Zhaobin Wang contributed to the conceptualization, methodology, software development, and validation. Zhen Yang contributed to the method, software development, and investigation. Jiajia Zhang supervised the project and reviewed and edited the manuscript. Huale Li and Shuhan Qi acquired funding and contributed to the review and editing of the manuscript.

Funding This work was funded by the National Natural Science Foundation of China (No.62406251), Natural Science Foundation of Gansu (No.26JRRA256), NSF of Guangdong (No. 2024A1515030024), Research Projects of Shenzhen (No. ZDCY20250901100302003), Shenzhen Science and Technology Program (No. KJZD20230923114213027).

Data availability No datasets were generated or analysed during the current study.

Code availability Codes are available when requesting.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Abou Risk N, Szafron D, et al (2010) Using counterfactual regret minimization to create competitive multi-player poker agents. In: AAMAS, pp. 159–166
- Basak A (2016) Abstraction using analysis of subgames. In: Proceedings of the 30th AAAI Conference on Artificial Intelligence, pp. 4196–4197. AAAI Press, Phoenix, USA
- Basilico N, Celli A, De Nittis G, Gatti N (2017) Computing the team-maxmin equilibrium in single-team single-adversary team games. *Intell Artif* 11(1):67–79
- Basilico N, Celli A, De Nittis G, Gatti N (2017) Team-maxmin equilibrium: efficiency bounds and algorithms. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31
- Berner C, Brockman G, Chan B, Cheung V, Debiak P, Dennison C, Farhi D, Fischer Q, Hashme S, Hesse C, et al (2019) Dota 2 with large scale deep reinforcement learning. *arXiv preprint [arXiv:1912.06680](https://arxiv.org/abs/1912.06680)*
- Blashfield RK, Aldenderfer MS (1988) The methods and problems of cluster analysis. *Handbook of Multivariate Experimental Psychology*, 447–473
- Bowling M, Burch N, Johanson M, Tammelin O (2015) Heads-up limit hold em poker is solved. *Science* 347(6218):145–149
- Brown GW (1951) Iterative solution of games by fictitious play. *Act Anal Prod Allocation* 13(1):374
- Brown N, Sandholm T (2018) Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science* 359(6374):418–424
- Brown N, Sandholm T (2019) Superhuman ai for multiplayer poker. *Science* 365(6456):885–890
- Brown N, Ganzfried S, Sandholm T (2015) Hierarchical abstraction, distributed equilibrium computation, and post-processing, with application to a champion no-limit texas hold'em agent. In: Proceedings of the 14th International Conference on Autonomous Agents and Multiagent Systems, pp. 7–15. ACM, Istanbul, Turkey

- Brown N, Lerer A, Gross S, Sandholm T (2019) Deep counterfactual regret minimization. In: Proceedings of the 36th International Conference on Machine Learning, pp. 793–802. PMLR, Long Beach, USA
- Brown N, Sandholm T (2015) Regret-based pruning in extensive-form games. In: Proceedings of the 28th International Conference on Neural Information Processing Systems, pp. 1972–1980. Curran Associates, Inc., Montreal, Canada
- Brown N, Sandholm T (2016) Strategy-based warm starting for regret minimization in games. In: Proceedings of the 30th AAAI Conference on Artificial Intelligence, pp. 432–438. AAAI Press, Arizona, USA
- Brown N, Sandholm T (2017) Safe and nested endgame solving for imperfect-information games. In: Proceedings of the 31st AAAI Conference on Artificial Intelligence, pp. 295–303. AAAI Press, San Francisco, USA
- Brown N, Sandholm T (2019) Solving imperfect-information games via discounted regret minimization. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 1829–1836
- Burch N, Moravcik M, Schmid M (2019) Revisiting cfr+ and alternating updates. *J Artif Intell Res* 64:429–443
- Burch N, Johanson M, Bowling M (2014) Solving imperfect information games using decomposition. In: Proceedings of the 28th AAAI Conference on Artificial Intelligence, pp. 602–608. AAAI Press, Québec, Canada
- Chen Y, Zhang L, Li S, Chen X, Pan G, Pan Z (2023) Rm-fsp: Regret minimization optimizes neural fictitious self-play. *Neurocomputing* 549:126471
- Chen Y, Deng X, Li C, Mguni D, Wang J, Yan X, Yang Y (2022) On the convergence of fictitious play: A decomposition approach. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, pp. 179–185. IJCAI: International Joint Conferences on Artificial Intelligence Organization
- Chen J, Lan T, Aggarwal V (2023) Hierarchical deep counterfactual regret minimization. *arXiv preprint arXiv:2305.17327*
- Chen J, Xie W, Zhang W, Wen Y, et al (2024) Offline fictitious self-play for competitive games. *arXiv e-prints*, 2403
- Coulom R (2006) Efficient selectivity and backup operators in monte-carlo tree search. In: International Conference on Computers and Games, pp. 72–83. Springer
- Daskalakis C, Goldberg PW, Papadimitriou CH (2009) The complexity of computing a nash equilibrium. *Commun ACM* 52(2):89–97
- D'Orazio R, Morrill D, Wright JR, Bowling M (2019) Alternative function approximation parameterizations for solving games: An analysis of f -regression counterfactual regret minimization. *arXiv preprint arXiv:1912.02967*
- Dushatskiy A, Chebykin A, Alderliesten T, Bosman P (2023) Multi-objective population based training. In: International Conference on Machine Learning, pp. 8969–8989. PMLR
- Ecoffet A, Huizinga J, Lehman J, Stanley KO, Clune J (2021) First return, then explore. *Nature* 590(7847):580–586
- Farina G, Kroer C, Sandholm T (2017) Regret minimization in behaviorally-constrained zero-sum games. In: International Conference on Machine Learning, pp. 1107–1116. PMLR
- Farina G, Kroer C, Sandholm T (2020) Stochastic regret minimization in extensive-form games. In: International Conference on Machine Learning, pp. 3018–3028. PMLR
- Farina G, Kroer C, Sandholm T (2021) Faster game solving via predictive blackwell approachability: Connecting regret matching and mirror descent. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 5363–5371
- Farina G, Schmucker R, Sandholm T (2020) Counterfactual-free regret minimization for sequential decision making and extensive-form games. In: AAAI Workshop on Reinforcement Learning in Games
- Ferguson TS (1970) On a class of infinite games related to liar dice. *The Annals of Mathematical Statistics*, 353–362
- Foster DP, Vohra R (1999) Regret in the on-line decision problem. *Games Econom Behav* 29(1–2):7–35
- Ganzfried S, Sandholm T (2013) Action translation in extensive-form games with large action spaces: Axioms, paradoxes, and the pseudo-harmonic mapping. In: Proceedings of the 23rd International Joint Conference on Artificial Intelligence, pp. 120–128. ijcai.org, Beijing, China
- Genugten B (2000) A weakened form of fictitious play in two-person zero-sum games. *Int Game Theory Rev* 2(04):307–328
- Gibson RG, Burch N, Lanctot M, Szafron D (2012) Efficient monte carlo counterfactual regret minimization in games with many player actions. In: Proceedings of the 25th International Conference on Neural Information Processing Systems, pp. 1880–1888. Curran Associates, Inc., Nevada, USA
- Gibson R, Lanctot M, Burch N, Szafron D, Bowling M (2012) Generalized sampling and variance in counterfactual regret minimization. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 26, pp. 1355–1361
- Gilpin A, Sandholm T (2007) Lossless abstraction of imperfect information games. *J ACM* 54(5):25–55

- Gilpin A, Sandholm T (2008) Expectation-based versus potential-aware automated abstraction in imperfect information games: an experimental comparison using poker. In: *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pp. 1454–1457. AAAI Press, Chicago, USA
- Goldstein M, Brown N (2022) Converging to unexploitable policies in continuous control adversarial games. In: *Deep Reinforcement Learning Workshop NeurIPS 2022*
- Hajibaba M, Gorgin S (2014) A review on modern distributed computing paradigms: Cloud computing, jungle computing and fog computing. *J Comput Inf Technol* 22(2):69–84
- Hansen KA, Hansen TD, Miltersen PB, Sørensen TB (2008) Approximability and parameterized complexity of minmax values. In: *Internet and Network Economics: 4th International Workshop, WINE 2008, Shanghai, China, December 17–20, 2008. Proceedings 4*, pp. 684–695. Springer
- Hawkin JA, Holte R, Szafron D (2011) Automated action abstraction of imperfect information extensive-form games. In: *Proceedings of the 25th AAAI Conference on Artificial Intelligence*, pp. 681–687. AAAI Press, San Francisco, USA
- He K, Wu H, Wang Z, Li H (2022) Finding nash equilibrium for imperfect information games via fictitious play based on local regret minimization. *Int J Intell Syst* 37(9):6152–6167
- Heinrich J, Lanctot M, Silver D (2015) Fictitious self-play in extensive-form games. In: *International Conference on Machine Learning*, pp. 805–813. PMLR
- Heinrich J, Silver D (2016) Deep reinforcement learning from self-play in imperfect-information games. arXiv preprint [arXiv:1603.01121](https://arxiv.org/abs/1603.01121)
- Hendon E, Jacobsen HJ, Sloth B (1996) Fictitious play in extensive form games. *Games Econom Behav* 15(2):177–202
- Hennes D, Morrill D, Omidshafiei S, Munos R, Perolat J, Lanctot M, Gruslys A, Lespiau J-B, Parmas P, Duéñez-Guzmán E, et al (2020) Neural replicator dynamics: Multiagent learning via hedging policy gradients. In: *Proceedings of the 19th International Conference on Autonomous Agents and Multiagent Systems*, pp. 492–501
- Hofbauer J, Sandholm WH (2002) On the global convergence of stochastic fictitious play. *Econometrica* 70(6):2265–2294
- Jaderberg M, Czarnecki WM, Dunning I, Marris L, Lever G, Castaneda AG, Beattie C, Rabinowitz NC, Morcos AS, Ruderman A et al (2019) Human-level performance in 3d multiplayer games with population-based reinforcement learning. *Science* 364(6443):859–865
- Jin P, Keutzer K, Levine S (2018) Regret minimization for partially observable deep reinforcement learning. In: *Proceedings of the 35th International Conference on Machine Learning*, pp. 2347–2356. PMLR, Stockholm, Sweden
- Johanson M, Burch N, Valenzano R, Bowling M (2013) Evaluating state-space abstractions in extensive-form games. In: *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems*, pp. 271–278. ACM, Saint Paul, USA
- Johanson M, Waugh K, Bowling M, Zinkevich M (2011) Accelerating best response calculation in large extensive games. In: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence*, pp. 258–265. ijcai.org, Barcelona, Spain
- Johanson M, Zinkevich M, Bowling M (2007) Computing robust counter-strategies. *Adv Neural Inf Process Syst* 20:15
- Ju Q, Hei F, Fang Z, Luo Y (2024) Integrating dynamic weighted approach with fictitious play and pure counterfactual regret minimization for equilibrium finding. arXiv preprint [arXiv:2402.12164](https://arxiv.org/abs/2402.12164)
- Keith A, Ahner D (2021) Counterfactual regret minimization for integrated cyber and air defense resource allocation. *Eur J Oper Res* 292(1):95–107
- Kim J (2024) Gpu-accelerated counterfactual regret minimization. arXiv preprint [arXiv:2408.14778](https://arxiv.org/abs/2408.14778)
- Kuhn HW (1950) A simplified two-person poker. *Contri Theor Games* 1(97–103):2
- Lambert Iii TJ, Epelman MA, Smith RL (2005) A fictitious play approach to large-scale optimization. *Oper Res* 53(3):477–489
- Lanctot M, Lisy V, Bowling M (2014) Search in imperfect information games using online monte carlo counterfactual regret minimization. In: *Proceedings of the 28th AAAI Conference on Artificial Intelligence*, pp. 34–41. AAAI Press, Québec, Canada
- Lanctot M, Lockhart E, Lespiau J-B, Zambaldi V, Upadhyay S, Pérolat J, Srinivasan S, Timbers F, Tuyls K, Omidshafiei S, et al (2019) OpenSpiel: A framework for reinforcement learning in games. arXiv preprint [arXiv:1908.09453](https://arxiv.org/abs/1908.09453)
- Lanctot M, Waugh K, Zinkevich M, Bowling M (2009) Monte carlo sampling for regret minimization in extensive games. In: *Proceedings of the 22nd International Conference on Neural Information Processing Systems*, pp. 1078–1086. Curran Associates, Inc., Vancouver, Canada
- LeCun Y, Bengio Y, Hinton G (2015) Nature. Deep learning 521(7553):436–444
- Leslie DS, Collins EJ (2006) Generalised weakened fictitious play. *Games Econom Behav* 56(2):285–298

- Li H, Wang X, Jia F, Li Y, Chen Q (2021) A survey of nash equilibrium strategy solving based on cfr. *Arch Comput Methods Eng* 28:2749–2760
- Li H, Guo Z, Liu Y, Wang X, Qi S, Zhang J, Xiao J (2023) Kdb-d2cfr: Solving multiplayer imperfect-information games with knowledge distillation-based deepcfr. *Knowl-Based Syst* 272:110567
- Li H, Huang W, Duan Z, Mguni DH, Shao K, Wang J, Deng X (2024) A survey on algorithms for nash equilibria in finite normal-form games. *Comput Sci Rev* 51:100613
- Li H, Wang X, Guo Z, Zhang J, Qi S (2024) D2cfr: minimize counterfactual regret with deep dueling neural network. *IEEE Trans Neural Netw Learn Syst* 35(12):18343–18356
- Li H, Hu K, Zhang S, Qi Y, Song L (2019) Double neural counterfactual regret minimization. In: *Proceedings of the 8th International Conference on Learning Representations*, pp. 1–13. OpenReview.net, Addis Ababa, Ethiopia
- Li J, Koyamada S, Ye Q, Liu G, Wang C, Yang R, Zhao L, Qin T, Liu T-Y, Hon H-W (2020) Suphx: Mastering mahjong with deep reinforcement learning. *arXiv preprint arXiv:2003.13590*
- Li A, Spyra O, Perel S, Dalibard V, Jaderberg M, Gu C, Budden D, Harley T, Gupta P (2019) A generalized framework for population based training. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1791–1799
- Lisy V, Davis T, Bowling M (2016) Counterfactual regret minimization in sequential security games. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30
- Lisy V, Lanctot M, Bowling MH (2015) Online monte carlo counterfactual regret minimization for search in imperfect information games. In: *AAMAS*, pp. 27–36
- Liu W, Jiang H, Li B, Li H (2022) Equivalence analysis between counterfactual regret minimization and online mirror descent. In: *International Conference on Machine Learning*, pp. 13717–13745. PMLR
- Liu W, Li B, Togelius J (2020) Model-free neural counterfactual regret minimization with bootstrap learning. *ArXiv Preprint arXiv:2012.01870*, 1–19
- Li S, Zhang Y, Wang X, Xue W, An B (2021) Cfr-mix: Solving imperfect information extensive-form games with combinatorial action space. *arXiv preprint arXiv:2105.08440*
- Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G, et al (2015) Human-level control through deep reinforcement learning. *nature* 518(7540), 529–533
- Moravčík M, Schmid M, Burch N, Lisý V, Morrill D, Bard N, Davis T, Waugh K, Johanson M, Bowling M (2017) Deepstack: expert-level artificial intelligence in heads-up no-limit poker. *Science* 356(6337):508–513
- Nash JF, et al (1950) Non-cooperative games
- Neller TW, Lanctot M (2013) An introduction to counterfactual regret minimization. In: *Proceedings of Model AI Assignments, the Fourth Symposium on Educational Advances in Artificial Intelligence (EAAI-2013)*, vol. 11
- Osborne MJ, Rubinstein A (1994) *A course in game theory*. MIT Press, Cambridge, MA
- Panageas I, Patris N, Skoulakis S, Cevher V (2024) Exponential lower bounds for fictitious play in potential games. *Advances in Neural Information Processing Systems* 36
- Perolat J, De Vylder B, Hennes D, Tarassov E, Strub F, Boer V, Muller P, Connor JT, Burch N, Anthony T et al (2022) Mastering the game of stratego with model-free multiagent reinforcement learning. *Science* 378(6623):990–996
- Perolat J, Piot B, Pietquin O (2018) Actor-critic fictitious play in simultaneous move multistage games. In: *International Conference on Artificial Intelligence and Statistics*, pp. 919–928. PMLR
- Perrin S, Pérolat J, Laurière M, Geist M, Elie R, Pietquin O (2020) Fictitious play for mean field games: Continuous time analysis and applications. *Adv Neural Inf Process Syst* 33:13199–13213
- Ponsen M, De Jong S, Lanctot M (2011) Computing approximate nash equilibria and robust best-responses using sampling. *J Artif Intell Res* 42:575–605
- Qin R-J, Pang J-C, Yu Y (2019) Improving fictitious play reinforcement learning with expanding models. *arXiv preprint arXiv:1911.11928*
- Qiu C, Wang X, Ma T, Wen Y, Zhang J (2024) Combining counterfactual regret minimization with information gain to solve extensive games with unknown environments. *Int J Intell Syst* 2024(1):9482323
- Schmid M, Burch N, Lanctot M, Moravčík M, Kadlec R, Bowling M (2019) Variance reduction in monte carlo counterfactual regret minimization (vr-mccfr) for extensive form games using baselines. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 2157–2164
- Schrittwieser J, Antonoglou I, Hubert T, Simonyan K, Sifre L, Schmitt S, Guez A, Lockhart E, Hassabis D, Graepel T et al (2020) Mastering atari, go, chess and shogi by planning with a learned model. *Nature* 588(7839):604–609
- Shapiro A (2003) Monte carlo sampling methods. *Handbooks Oper Res Management Sci* 10:353–425
- Shi J, Littman ML (2000) Abstraction methods for game theoretic poker. In: *Proceedings of the 2nd International Conference on Computers and Games*, pp. 333–345. Springer, Hamamatsu, Japan

- Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, Lanctot M, Sifre L, Kumaran D, Graepel T et al (2018) A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science* 362(6419):1140–1144
- Silver D, Huang A, Maddison CJ, Guez A, Sifre L, Van Den Driessche G, Schrittwieser J, Antonoglou I, Panneershelvam V, Lanctot M, et al (2016) Mastering the game of go with deep neural networks and tree search. *nature* 529(7587), 484–489
- Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, Hubert T, Baker L, Lai M, Bolton A, et al (2017) Mastering the game of go without human knowledge. *nature* 550(7676), 354–359
- Southey F, Bowling M, Larson B, Piccione C, Burch N, Billings D, Rayner C (2005) Bayes' bluff: opponent modelling in poker. In: *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, pp. 550–558
- Steinberger E (2019) Single deep counterfactual regret minimization. *ArXiv Preprint* [arXiv:1901.07621](https://arxiv.org/abs/1901.07621), 1–13
- Steinberger E, Lerer A, Brown N (2021) Dream: Deep regret minimization with advantage baselines and model-free learning. In: *Proceedings of the 35th AAAI Conference on Artificial Intelligence*, pp. 5300–5308. AAAI Press, Virtual Event
- Sutton RS, Barto AG (2018) *Reinforcement Learning: An Introduction*. MIT press
- Swaminathan A, Joachims T (2015) Counterfactual risk minimization: Learning from logged bandit feedback. In: *International Conference on Machine Learning*, pp. 814–823. PMLR
- Sychrovský D, Sustr M, Bowling M, Schmid M (2023) Global cfr: Meta-learning in self-play regret minimization. In: *OPT 2023: Optimization for Machine Learning*
- Tammelin O (2014) Solving large imperfect information games using cfr+. *ArXiv Preprint* [arXiv:1407.5042](https://arxiv.org/abs/1407.5042), 1–6
- Teófilo LFG, Reis LP, Cardoso HL (2013) Speeding-up poker game abstraction computation: Average rank strength. In: *Proceedings of the 27th AAAI Conference on Artificial Intelligence*, pp. 56–64. AAAI Press, Bellevue, USA
- Vinyals O, Babuschkin I, Czarnecki WM, Mathieu M, Dudzik A, Chung J, Choi DH, Powell R, Ewalds T, Georgiev P, et al (2019) Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature* 575(7782), 350–354
- Von Stengel B, Koller D (1997) Team-maxmin equilibria. *Games Econom Behav* 21(1–2):309–321
- Wang Y, Sun P, Zhang D, Li Y (2025) Counterfactual regret minimization for the safety verification of autonomous driving. *Appl Intell* 55(5):312
- Wang S, Wang J, Song B (2025) Horse-cfr: Hierarchical opponent reasoning for safe exploitation counterfactual regret minimization. *Expert Syst Appl* 263:125697
- Watkins CJ, Dayan P (1992) Q-learning. *Mach Learn* 8:279–292
- Waugh K, Morrill D, Bagnell JA, Bowling M (2015) Solving games with functional regret estimation. In: *Proceedings of the 29th AAAI Conference on Artificial Intelligence*, pp. 2138–2145. AAAI Press, Austin, USA
- Xu Z, Yu C, Liang Y, Wu Y, Wang Y (2025) Learning global nash equilibrium in team competitive games with generalized fictitious cross-play. *J Mach Learn Res* 26(44):1–30
- Xu Z, Liang Y, Yu C, Wang Y, Wu Y (2023) Fictitious cross-play: Learning global nash equilibrium in mixed cooperative-competitive games. In: *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pp. 1053–1061
- Xu H, Li K, Fu H, Fu Q, Xing J (2022) Autocfr: Learning to design counterfactual regret minimization algorithms. In: *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, pp. 10–18. AAAI Press, Virtual Event, Canada
- Xu H, Li K, Fu H, Fu Q, Xing J, Cheng J (2024) Dynamic discounted counterfactual regret minimization. In: *The Twelfth International Conference on Learning Representations*
- Xu H, Li K, Liu B, Fu H, Fu Q, Xing J, Cheng J (2024) Minimizing weighted counterfactual regret with optimistic online mirror descent. *arXiv preprint* [arXiv:2404.13891](https://arxiv.org/abs/2404.13891)
- Yang Y, Wang J (2020) An overview of multi-agent reinforcement learning from game theoretical perspective. *arXiv preprint* [arXiv:2011.00583](https://arxiv.org/abs/2011.00583)
- Ye M, Han Q-L, Ding L, Xu S (2023) Distributed nash equilibrium seeking in games with partial decision information: A survey. *Proc IEEE* 111(2):140–157
- Zenati H, Diemert E, Martin M, Mairal J, Gaillard P (2023) Sequential counterfactual risk minimization. In: *International Conference on Machine Learning*, pp. 40681–40706. PMLR
- Zhang BH, Farina G, Sandholm T (2023) Team belief dag: generalizing the sequence form to team games for fast computation of correlated team max-min equilibria via regret minimization. In: *International Conference on Machine Learning*, pp. 40996–41018. PMLR

- Zhang L, Chen Y, Wang W, Han Z, Li S, Pan Z, Pan G (2021) A monte carlo neural fictitious self-play approach to approximate nash equilibrium in imperfect-information dynamic games. *Front Comp Sci* 15:1–14
- Zhang Y, An B (2020) Converging to team-maxmin equilibria in zero-sum multiplayer games. In: *International Conference on Machine Learning*, pp. 11033–11043. PMLR
- Zhang Y, An B, Subrahmanian V (2022) Correlation-based algorithm for team-maxmin equilibrium in multiplayer extensive-form games. In: *31st International Joint Conference on Artificial Intelligence, IJCAI 2022*, pp. 606–612. *International Joint Conferences on Artificial Intelligence*
- Zha D, Xie J, Ma W, Zhang S, Lian X, Hu X, Liu J (2021) Douzero: Mastering doudizhu with self-play deep reinforcement learning. In: *International Conference on Machine Learning*, pp. 12333–12344. PMLR
- Zhou Y, Ren T, Li J, Yan D, Zhu J (2019) Lazy-cfr: Fast and near-optimal regret minimization for extensive games with imperfect information. In: *Proceedings of the 8th International Conference on Learning Representations*, pp. 1–15. *OpenReview.net*, Addis Ababa, Ethiopia
- Zinkevich M, Johanson M, Bowling M, Piccione C (2007) Regret minimization in games with incomplete information. *Advances in neural information processing systems* 20

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.